CEH v13 AI (312-50) Study Guide - Module 10: Denial of Service

EC-Council CEH v13 AI (312-50) Exam Prep

Study guide built from official module screenshots

Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Topic-by-Topic Guide
   - 3.1 What is a DoS Attack [page 1]
   - 3.2 What is a DDoS Attack [page 1]
   - 3.3 Scanning Methods for Finding Vulnerable Machines [page 2]
   - 3.4 How Does Malicious Code Propagate [page 2]
   - 3.5 Use of Mobile Devices as Botnets for Launching DDoS Attacks [page 3]
   - 3.6 Basic Categories of DoS/DDoS Attack Vectors [page 3]
   - 3.7 UDP Flood Attack [page 4]
   - 3.8 ICMP Flood Attack [page 4]
   - 3.9 Ping of Death Attack [page 5]
   - 3.10 Smurf Attack [page 5]
   - 3.11 NTP Amplification Attack [page 5]
   - 3.12 SYN Flood Attack [page 6]
   - 3.13 Fragmentation Attack [page 6]
   - 3.14 Spoofed Session Flood Attack [page 7]
   - 3.15 HTTP GET/POST Attack [page 7]
   - 3.16 Slowloris Attack [page 7]
   - 3.17 Multi-Vector Attack [page 8]
   - 3.18 Permanent Denial-of-Service Attack [page 8]
   - 3.19 TCP SACK Panic Attack [pages 8 and 9]
   - 3.20 Distributed Reflection Denial-of-Service (DRDoS) Attack [page 9]
   - 3.21 DoS/DDoS Attack Toolkits in the Wild [page 10]
   - 3.22 Detection Techniques [page 10]
   - 3.23 DoS/DDoS Countermeasure Strategies [page 11]
   - 3.24 DDoS Attack Countermeasures [page 11]
   - 3.25 DDoS Attack Countermeasures: Prevent Potential Attacks [page 12]
   - 3.26 DDoS Attack Countermeasures: Deflect Attacks [page 12]
   - 3.27 Techniques to Defend against Botnets [page 13]
   - 3.28 DoS/DDoS Protection at ISP Level [page 13]
   - 3.29 DoS/DDoS Protection Tools [page 14]
   - 3.30 DoS/DDoS Protection Services [page 14]
   - 3.31 Module Summary, Botnet Ecosystem, and the Google Cloud HTTP/2 Rapid Reset Case Study [page 15]
   - 3.32 AI and DoS: AI-Powered DDoS and Attacks on AI Systems [not in the screenshots]

4. Tool Encyclopedia
    4.1 Attack tools
    4.2 Defense, detection, and protection tools
    4.3 Look-alike tools and how to pick
5. Command Sheet
    5.1 ping (Ping of Death and ICMP flood)
    5.2 hping3 (SYN flood, UDP flood, ICMP flood, spoofing)
    5.3 ntpdc (NTP monlist query, used in NTP amplification)
    5.4 Cisco IOS: ip tcp intercept (SYN flood defense)
    5.5 Cisco IOS: ip verify unicast source reachable-via (ingress filtering, uRPF)
6. Ports, Protocols, Standards and Acronyms
    6.1 Ports and protocols seen in the screenshots or implied by them
    6.2 Standards and acronyms
7. Attack vs Countermeasure and Look-alike Comparisons
    7.1 Attack and its countermeasure
    7.2 Look-alike pairs
8. Exam Cheat Sheet
    8.1 Keyword to answer
    8.2 Must-memorize lists
    8.3 Numbers and defaults
    8.4 Rules for picking the BEST option
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report
    12.1 Map of pages to guide sections
    12.2 Items marked [UNCLEAR]
    12.3 Items to double check
    12.4 Coverage confirmation

CEH v13 AI (312-50) Study Guide - Module 10: Denial of Service

Source: 15 pages of screenshots (29 slide images, 28 unique, one slide repeated) from the official Module 10 theory videos. Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry to read with full confidence. The guide follows the screenshots in page order so it lines up with the video. The module summary slide (page 15) lists a botnet ecosystem section and a Google Cloud HTTP/2 Rapid Reset case study. No screenshots of those two parts were supplied, so they are covered as ADDED CONTEXT in section 3.31.

1. Module at a Glance

Module: 10 - Denial-of-Service (EC-Council CEH v13 AI).

What this module is (5 lines):

1. A Denial-of-Service (DoS) attack makes a system, network, or service unavailable to its real users by overloading it. A Distributed DoS (DDoS) attack does the same from thousands of compromised machines (a botnet).
2. The module explains how a botnet is built: scanning for vulnerable machines, spreading malicious code, and enslaving phones and PCs.
3. Attacks are sorted into three families by what they exhaust: volumetric (bandwidth, measured in bps), protocol (state tables, measured in pps), and application layer (app resources, measured in rps).
4. Each named attack is covered: UDP flood, ICMP flood, Ping of Death, Smurf, NTP amplification, SYN flood, fragmentation, spoofed session flood, HTTP GET/POST, Slowloris, multi-vector, permanent DoS (phlashing), TCP SACK Panic, and DRDoS.
5. The module ends with attack tools, detection methods, a set of countermeasures (egress and ingress filtering, TCP Intercept, rate limiting, honeypots, black holes, ISP protection), and protection tools and services.

Why it matters: DoS is an availability attack. Of the CIA triad (Confidentiality, Integrity, Availability), it hits availability. It is cheap for attackers, hard to stop, and is often used as a smokescreen while another attack happens, or for extortion. In the five hacking phases it does not map neatly to one phase. It is usually a goal on its own, or a distraction during gaining access.

What the exam usually asks from it:

- Match a description to the attack: which flood, which family (volumetric, protocol, application layer), which unit measures it.
- How each attack works: spoofed source, broadcast address, monlist, listen queue, partial HTTP headers, integer overflow in the kernel.
- Which tool fits: LOIC, HOIC, HULK, Slowloris, UFONet, hping3, ISB, UltraDDOS-v2.

- Which countermeasure matches which attack: TCP Intercept for SYN flood, ingress filtering for spoofing, rate limiting for floods, black hole filtering, DDoS-protection services.
- The three detection techniques: activity profiling, sequential change-point detection (Cusum), wavelet-based signal analysis.
- The three botnet propagation methods and five scanning methods.
- Scenario questions: "which is the BEST next step during an attack".

The 10 things to remember most:

1. DoS = one source, DDoS = many sources (botnet of zombies controlled by a Command and Control (C&C) server). The goal is lost availability.
2. Three attack families: Volumetric (bps), Protocol (pps), Application layer (rps). Know which unit goes with which family.
3. Volumetric attacks on the slides: UDP flood, ICMP flood, Ping of Death, Smurf, pulse wave, zero-day, NTP amplification.
4. Protocol attacks on the slides: SYN flood, fragmentation, spoofed session flood, ACK flood, TCP SACK Panic.
5. Application-layer attacks on the slides: HTTP GET/POST, Slowloris, UDP application layer flood, DDoS extortion.
6. Smurf = spoofed victim IP + ICMP echo to a broadcast address. NTP amplification = spoofed victim IP + monlist command. Both are reflection and amplification attacks.
7. SYN flood fills the listen queue (half-open connections held at least 75 seconds). Defense: TCP Intercept, SYN cookies (ADDED CONTEXT).
8. Ping of Death: packet larger than the 65,535-byte IP limit (RFC 791), crashes the target during reassembly.
9. Permanent DoS (phlashing) damages hardware by "bricking" it with a fraudulent firmware update. Replace the hardware to recover.
10. Defend in layers: filter spoofed traffic (ingress and egress, RFC 3704), rate limit, TCP Intercept, black hole, ISP and cloud scrubbing, honeypots, and patch to remove bots.

2. Big Picture Map

```
DENIAL OF SERVICE (Module 10)
|
+-- CONCEPTS
|    DoS -> DDoS (botnet, C&C, zombies)
|    -> Scanning methods -> Propagation methods -> Mobile botnets
|
+-- ATTACK FAMILIES
|    Volumetric (bps)  : UDP, ICMP, PoD, Smurf, NTP, pulse wave
|    Protocol (pps)    : SYN, fragmentation, spoofed session,
|                        ACK flood, TCP SACK Panic
|    Application (rps) : HTTP GET/POST, Slowloris, extortion
|    Special           : Multi-vector, Permanent DoS, DRDoS
|
+-- TOOLS AND DETECTION
|    ISB, UltraDDOS-v2, HOIC, LOIC, HULK, Slowloris, UFONet
|    Activity profiling, Change-point (Cusum), Wavelet analysis
|
+-- COUNTERMEASURES
     Strategies (absorb, degrade, shut down)
     -> Prevent (egress, ingress, TCP Intercept, rate limiting)
     -> Deflect (honeypots) -> Botnet defense (RFC 3704, black hole)
     -> ISP level -> Protection tools and services
```

Read top to bottom: learn what DoS and DDoS are and how a botnet is built, then learn each attack by the resource it exhausts, then the tools attackers use and how defenders detect the traffic, and finally the layered defenses.

The three families side by side:

Family	What it exhausts	Unit	Typical example
Volumetric	Bandwidth	bps	UDP flood, Smurf
Protocol	State tables, connections	pps	SYN flood
Application layer	App or service resources	rps	HTTP GET flood

3. Topic-by-Topic Guide

3.1 What is a DoS Attack [page 1]

What it is (simple words): A Denial-of-Service (DoS) attack is an attack on a computer or network that reduces, restricts, or prevents access to system resources for its legitimate users. The attacker floods the victim with non-legitimate service requests or traffic so the system runs out of capacity. The slide lists four impacts: loss of goodwill (customers lose trust), network outages, financial losses, and operational disruptions.

Analogy: A shop has one door. A crowd of fake customers blocks the door and asks pointless questions. Real customers cannot get in, so the shop loses sales even though nothing inside was stolen or broken.

How it works (numbered steps):

1. The attacker picks a target resource: bandwidth, CPU, memory, a connection table, or an application.
2. The attacker sends far more traffic or requests than the target can handle, or sends traffic crafted to waste resources.
3. The target spends its capacity on the attack traffic.
4. Legitimate requests are slow, dropped, or refused.
5. The service goes down or becomes unusable until the attack stops or is filtered.

The slide diagram shows this: malicious traffic from several attackers and regular traffic from a normal user both travel through the Internet and a router toward a server cluster. The caption says the malicious traffic consumes all the available bandwidth, so regular traffic cannot get through.

```
Attacker(s) --attack traffic--+
                              v
                  Internet -> Router -> Server cluster
                              ^
Regular user --regular traffic+
(attack traffic fills the pipe, regular traffic is squeezed out)
```

Real-world use: A competitor pays a "booter" service to hit an online shop on a sale day. The shop's link is saturated, checkout pages time out, and the shop loses a day of revenue and customer trust. The attacker never needed to break in.

Tools and commands used here: None on this slide. Tools are in section 3.21 and commands in section 5.

AI angle: Attackers can use AI to automate target selection and to adapt traffic patterns so they look more human and avoid simple filters. Defenders use machine-learning anomaly detection to separate attack traffic from normal spikes. Example prompt (defender): "Here are 5-minute NetFlow summaries for the last hour. Flag the intervals where packets per second and unique source IPs rise abnormally and say which DoS family the pattern looks like."

Countermeasures and defense: Covered in sections 3.22 to 3.30. In short: detect the abnormal rise, filter spoofed traffic, rate limit, absorb with extra capacity, and use an ISP or cloud scrubbing service.

EXAM FOCUS:

- Keywords "reduces, restricts, or prevents accessibility" and "legitimate users" -> DoS definition.

- DoS attacks the A in the CIA triad (Availability). It does not steal data.
- Trap: a DoS attack does not need to "hack in". Questions may say "no data was stolen", and the answer is still DoS.
- Look-alike: DoS (one source) vs DDoS (many sources). See section 3.2.

Memory trick: "Door jammed, not door broken." The system still works. It is just too busy to serve you.

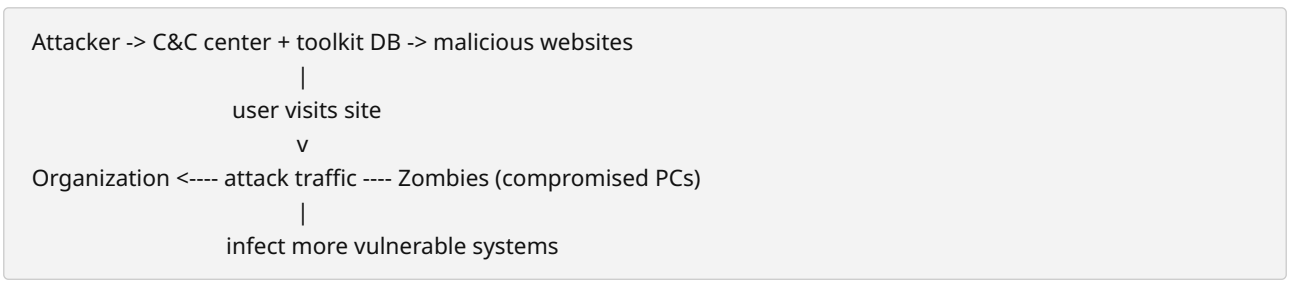
3.2 What is a DDoS Attack [page 1]

What it is (simple words): A Distributed Denial-of-Service (DDoS) attack is a DoS attack launched from many compromised machines at the same time. The compromised machines are called bots or zombies. Together they form a botnet. The attacker controls them through a Command and Control (C&C) server. Because the traffic comes from thousands of different real IP addresses, it is much bigger and much harder to block than a single-source attack.

Analogy: One person shouting at a shop door is easy to remove. Ten thousand people, each paid to walk in and block the door, is a different problem. Security cannot remove them all, because each one looks like a real customer.

How it works (numbered steps): The slide shows a 12-step ecosystem. The small text was hard to read, so steps 8 to 11 are a best reading [UNCLEAR on page 1].

1. The attacker sets up a C&C center and a Crimeware Toolkit database.
2. The attacker recruits affiliates into an affiliation network.
3. Affiliates contribute malware and release the DDoS toolkit.
4. Malicious websites redirect users to the Crimeware toolkit database.
5. Attackers compromise a legitimate website or create a new malicious website, using phishing and social engineering to lure victims.
6. Users visit the compromised legitimate website or the malicious site.
7. The malicious website redirects the user to the Crimeware toolkit server.
8. Malware infects the user systems, which become Compromised PCs (zombies) [UNCLEAR on page 1: exact wording].
9. Bots connect back to the C&C center [UNCLEAR on page 1].
10. Bots receive instructions from the C&C center to attack the primary target [UNCLEAR on page 1].
11. Infected systems look for other vulnerable systems and infect them, which grows the botnet.
12. The botnet attacks the primary target (the organization).



Real-world use: A criminal group rents a botnet of 50,000 infected home routers. The buyer gives the C&C server a target IP and a time. All bots send traffic together. The victim sees traffic from tens of thousands of ordinary home IPs and cannot simply block one address.

Tools and commands used here: Botnet toolkits and DDoS tools (section 3.21). C&C servers are the control point.

AI angle: AI can write phishing lures for step 5 at scale and in many languages. Attackers also use AI to tune bot behavior so traffic looks like real browsing. Defenders use AI to cluster bot behavior. Example prompt (defender): "Group these 10,000 source IPs by request rate, user agent, and timing, and list the clusters that look like one botnet."

Countermeasures and defense: Protect secondary victims (patch, anti-virus, awareness), detect and neutralize handlers, filter spoofed addresses, and use ISP or cloud protection (sections 3.24 to 3.30).

EXAM FOCUS:

- Keywords: "bots", "zombies", "C&C", "botnet" -> DDoS.
- A handler (master) controls agents (zombies, bots). Few handlers control many agents, so removing handlers is efficient (see 3.24).
- Trap: the "primary victim" is the target. "Secondary victims" are the compromised machines used to attack it.
- Look-alike: DDoS (many bots, direct) vs DRDoS (reflected through innocent servers, section 3.20).

Memory trick: "Recruit, Infect, Command, Attack." The attacker recruits helpers, infects victims, commands the bots, and attacks.

3.3 Scanning Methods for Finding Vulnerable Machines [page 2]

What it is (simple words): After a machine is infected, it often searches for more vulnerable machines to infect. That is how a botnet grows by itself, like a worm. The slide lists five scanning methods. The labels of rows 2 and 3 were partly hidden by the presenter, so those two names come from the matching descriptions [UNCLEAR on page 2].

Method	How it finds targets
Random scanning	Probes random IPs in the target range
Hit-list scanning	Scans a prepared list of likely-vulnerable hosts
Topological scanning	Uses data found on the infected machine
Local subnet scanning	Looks in its own local network
Permutation scanning	Walks a pseudorandom permutation of IPs

Analogy: A burglar looking for unlocked houses. Random: tries doors anywhere. Hit-list: has a written list of houses with weak locks. Topological: reads the neighbor addresses from the

victim's address book. Local subnet: tries every house on the same street. Permutation: uses a shuffled map so no two burglars try the same house.

How it works (numbered steps):

1. Random scanning: the infected machine probes IP addresses chosen at random from the target network IP range and checks each for the vulnerability.
2. Hit-list scanning: the attacker first collects a list of potentially vulnerable machines, then scans them to find the vulnerable ones. This is fast because it skips empty address space.
3. Topological scanning: it uses information obtained from an infected machine (for example, email contacts, host files, ARP cache, or connection history) to find new vulnerable machines.
4. Local subnet scanning: the infected machine looks for new vulnerable machines in its own local network. It sits behind the firewall, so it can reach hosts that outside scanners cannot.
5. Permutation scanning: it uses a pseudorandom permutation list of IP addresses. All infected machines share the same shuffled list but start at different points. Each finds new hosts without re-scanning hosts that are already infected. When a machine meets an already infected host, it can stop or restart at a new point (ADDED CONTEXT).

Real-world use: A worm lands on a laptop inside a company LAN. It uses local subnet scanning to find unpatched file servers on the same network. It also reads the laptop's recent connections for topological scanning. Within an hour, dozens of internal machines are enslaved.

Tools and commands used here: None on the slide. Real scanners such as nmap (Module 3) do the same job for defenders and testers.

AI angle: An attacker can ask an AI model to write a scanning script. A defender can feed scan logs to AI to spot internal scanning (many connection attempts to many hosts on one port). Example prompt: "Write a Python script that reads a text file of IPs and reports which ones have port 445 open. Use only the standard library." (Use only on systems you own or are authorized to test.)

Countermeasures and defense: Patch vulnerable services, segment the network (stops local subnet scanning), block unused ports, run egress filtering, and use IDS/IPS to flag scan patterns.

EXAM FOCUS:

- Keywords: "pseudorandom permutation list" -> permutation scanning.
- Keywords: "list of potentially vulnerable machines first" -> hit-list.
- Keywords: "own local network" -> local subnet scanning.
- Keywords: "information obtained from an infected machine" -> topological scanning.
- Keywords: "randomly from the target IP range" -> random scanning.

- Trap: random and permutation both look random. Permutation uses a shared shuffled list, so it avoids repeats.

Memory trick: "R-H-T-L-P": Random, Hit-list, Topological, Local subnet, Permutation. Say "Rapid Hackers Take Local Pizza."

3.4 How Does Malicious Code Propagate [page 2]

What it is (simple words): Once a vulnerable machine is found, the attacker needs to deliver the attack toolkit (the malicious code) to it. The slide shows three techniques: central source propagation, back-chaining propagation, and autonomous propagation. The difference is who sends the toolkit to the new victim.

Technique	Who sends the toolkit to the new victim
Central source	A central server (the central source)
Back-chaining	The attacker's own system, through the victim chain
Autonomous	The attacking host itself, at the moment of break-in

Analogy: Spreading a flyer. Central source: everyone picks up flyers from one shop. Back-chaining: each new person comes back to the original creator to get a copy. Autonomous: the person who just recruited you hands you the flyer on the spot.

How it works (numbered steps):

Central source propagation:

1. The attacker places the attack toolkit on a central source (a server or website).
2. The attacker exploits a vulnerable system (the victim).
3. A copy of the toolkit is transferred from the central source to the victim.
4. The victim repeats the process on the next victim.

Back-chaining propagation:

1. The attacker places the attack toolkit on his or her own system.
2. The attacker exploits a vulnerable system.
3. A copy of the toolkit is transferred from the attacker's system to the newly discovered vulnerable system (the connection goes back to the machine that attacked it, hence "back-chaining").
4. The new victim repeats the process.

Autonomous propagation:

1. The attacker exploits a vulnerable system and the attacking host itself transfers the toolkit to the new victim at the exact time it breaks into that system.
2. There is no separate download step. The exploit and the copy happen together.
3. The new victim repeats the process on the next victim.

```
CENTRAL SOURCE     Attacker -> Victim -> Next victim
                   ^       |
                   +-- Central source (copy code) --+

BACK-CHAINING      Attacker (holds toolkit) <- pulls from -> Victim

AUTONOMOUS         Attacker -- exploit + copy code --> Victim
                Victim   -- exploit + copy code --> Next victim
```

Real-world use: A worm that carries its payload inside the exploit packet is autonomous: it infects and copies in one step, so it spreads very fast. A botnet where each new bot downloads a payload from one web server is central source: it is easier to track and take down because there is a single server to block.

Tools and commands used here: None on the slide.

AI angle: AI can help malware authors mutate payloads so that each copy has a different signature, which makes signature detection weaker. Defenders use behavior-based AI detection instead. Example prompt (defender): "Summarize which hosts downloaded files from the same internal IP within one minute of each other, as a possible propagation chain."

Countermeasures and defense: Block known bad central sources with reputation filtering, apply egress filtering so infected hosts cannot download toolkits, patch exploitable services, and use EDR and anti-malware.

EXAM FOCUS:

- Keywords: "attack toolkit on the central source" -> central source.
- Keywords: "own system" and "copy transferred to the newly discovered system" -> back-chaining.
- Keywords: "at the exact time that it breaks into" -> autonomous.
- Trap: central source and back-chaining both use a place that holds the toolkit. Central source = a separate server. Back-chaining = the attacker's own system.
- Count: three propagation techniques; five scanning methods.

Memory trick: "Central = a Shop, Back-chain = go Back to the boss, Autonomous = Automatic hand-off."

3.5 Use of Mobile Devices as Botnets for Launching DDoS Attacks [page 3]

What it is (simple words): Phones and tablets can be turned into bots too. Android devices are passively vulnerable to malware such as Trojans, bots, and Remote Access Trojans (RATs), which are often found in third-party app stores. Unsecured Android devices are primary targets for attackers who want to enlarge their botnet, because they are highly vulnerable. Infection methods include malicious Android apps (even in the Google Play store) and drive-by downloads (software that installs just by visiting a web page).

Analogy: A pickpocket who plants a tiny remote-controlled device in a stranger's coat. The owner walks around normally. The device obeys the pickpocket whenever he sends a signal.

How it works (numbered steps):

1. The attacker builds or takes a malicious Android Package (APK) file. The slide says the attacker binds the malicious APK server to the APK file [UNCLEAR on page 3: this wording is odd; the usual meaning is that the malicious payload is bound into a normal app's APK].
2. The attacker encrypts the package and removes unwanted features and permissions so the app looks harmless.
3. The attacker distributes the package to a third-party app store (the slide gives the Google Play store as another example of a store).
4. The user is tricked into downloading and installing the app.
5. The attacker gains full control of the device and enslaves it into the attacker's mobile botnet.
6. The bot is used to launch DDoS attacks and web injections.

```
Attacker -> bind payload into APK -> encrypt -> remove permissions
    -> upload to app store -> user installs
    -> phone joins mobile botnet -> DDoS, web injections
```

Real-world use: A free "flashlight" app with a hidden bot is installed by 100,000 users. The C&C server later tells every phone to send HTTP requests to a bank's website at 9 pm. The bank sees traffic from 100,000 mobile IPs on many carriers.

Tools and commands used here: APK (Android Package file format). Related tools are in Module 17 (Hacking Mobile Platforms).

AI angle: AI can generate convincing app descriptions, fake reviews, and icons that make a malicious app look genuine. Defenders use AI-based app scanning and mobile threat defense. Example prompt (defender): "List the permissions in this app manifest that do not fit a flashlight app."

Countermeasures and defense: Install apps only from trusted stores, review requested permissions, keep Android patched, use mobile device management (MDM) and mobile threat defense, and disable installation from unknown sources.

EXAM FOCUS:

- Keywords: "APK", "third-party app store", "mobile botnet" -> mobile devices as bots.
- The botnet uses the phone for DDoS attacks and web injections.
- Trap: the slide says malicious apps can be in the official Google Play store too. Do not assume "official store = safe".
- Look-alike: drive-by download (infection by visiting a page) vs malicious app (infection by installing).

Memory trick: "Phone = pocket zombie."

3.6 Basic Categories of DoS/DDoS Attack Vectors [page 3]

What it is (simple words): An attack vector is the path or method used to deliver an attack. The slide sorts DoS and DDoS vectors into three categories by the resource they consume. Each category has its own unit of measure (the "magnitude") and its own list of techniques. This is one of the most testable slides in the module.

Category	Consumes	Measured in
Volumetric	Bandwidth of target network or service	bits per second (bps)
Protocol	Connection state tables and similar resources	packets per second (pps)
Application layer	Resources or services of an application	requests per second (rps)

Analogy: A restaurant. Volumetric: a mob fills the street so no one can reach the door (road capacity). Protocol: the host's reservation book is filled with fake bookings (state table). Application layer: one customer orders the most complicated dish over and over so the kitchen cannot serve anyone else (app resource).

How it works (numbered steps):

Volumetric attacks:

1. They consume the bandwidth of a target network or service.
2. Magnitude is measured in bits per second (bps).
3. Types of bandwidth depletion attacks: flood attacks and amplification attacks.
4. Techniques on the slide: UDP flood, ICMP flood, Ping of Death and Smurf, pulse wave and zero-day attack, NTP amplification.

Protocol attacks:

1. They consume other types of resources, such as the connection state tables found in network infrastructure components like load balancers, firewalls, and application servers.
2. Magnitude is measured in packets per second (pps).
3. Techniques on the slide: SYN flood, fragmentation attack, spoofed session flood, ACK flood, TCP SACK panic attack.

Application layer attacks:

1. They consume the resources or services of an application, so the application becomes unavailable to other legitimate users.
2. Magnitude is measured in requests per second (rps).
3. Techniques on the slide: HTTP GET/POST attack, Slowloris attack, UDP application layer flood attack, DDoS extortion attack.

ADDED CONTEXT (techniques that appear in the list but have no dedicated slide):

- ACK flood: the attacker sends a very large number of TCP packets with only the ACK flag set. The target must check each one against its connection table, find no matching connection, and respond with RST (or silently drop it). Stateful firewalls and load balancers spend CPU and table lookups on packets that belong to no session. Tool example: hping3 with the -A flag. Defense: stateful filtering, rate limiting, and dropping ACKs that match no session.
- Pulse wave attack: short, repeated bursts of very high-volume traffic with quiet gaps in between (for example, a burst every 10 minutes). The gaps are short, so the defender's mitigation never fully recovers before the next pulse. Defense: always-on cloud mitigation instead of on-demand switching.
- Zero-day DDoS attack: a DDoS method that uses a new, unpatched flaw or an unknown technique, so no signature exists yet. Example: HTTP/2 Rapid Reset (section 3.31) was a zero-day when first used. Defense: behavior-based detection, rate limiting, and vendor patches.
- UDP application layer flood: UDP packets sent to a specific application service (such as DNS or a game server) to exhaust that service rather than just the link. Defense: rate limiting per source and service-specific filtering.
- DDoS extortion attack (also called ransom DDoS or RDoS): the attacker threatens a DDoS attack, or runs a short demonstration attack, and demands payment to stop. Defense: do not pay, notify law enforcement, and use a scrubbing service.

Real-world use: A security team sees 80 Gbps of inbound traffic. They call it a volumetric attack because the problem is bits per second. A week later the link looks normal at 200 Mbps, but the web server is at 100 percent CPU from 40,000 requests per second for a search page. That is an application layer attack.

Tools and commands used here: LOIC, HOIC, HULK, Slowloris, hping3 (sections 3.21 and 5).

AI angle: AI-driven attackers can mix vectors automatically (see multi-vector, section 3.17). Defenders use AI to label the family of an attack from traffic features. Example prompt (defender): "Given these metrics: 90 Gbps, 8 Mpps, 12,000 rps, 0.5 percent SYN. Which attack family is dominant and why?"

Countermeasures and defense: Match the defense to the family. For volumetric attacks, add capacity and use ISP or cloud scrubbing. For protocol attacks, use TCP Intercept, SYN cookies, and stateful filtering. For application-layer attacks, use a Web Application Firewall (WAF), rate limiting, and CAPTCHA or bot challenges.

EXAM FOCUS:

- Units are a favorite: bps -> volumetric, pps -> protocol, rps -> application layer.
- Keywords: "connection state tables", "load balancers, firewalls" -> protocol attack.
- Keywords: "resources or services of an application" -> application layer attack.
- Trap: Ping of Death and Smurf sit under volumetric on this slide, even though Ping of Death exploits packet size. Follow the slide.

- Trap: TCP SACK Panic is listed under protocol attacks, even though it crashes the kernel.
- Look-alike: flood attack (attacker sends the volume) vs amplification attack (a third-party server multiplies the volume).

Memory trick: "B-P-R: Bandwidth-bps, Packets-pps, Requests-rps." Volumetric starts with "V" for "volume of bits". Protocol = Packets. Application = Requests.

3.7 UDP Flood Attack [page 4]

What it is (simple words): In a UDP flood, the attacker sends spoofed User Datagram Protocol (UDP) packets at a very high rate to random ports of a target server, using a large source IP range. UDP is connectionless: there is no handshake, so the target cannot tell if the sender is real. The flood consumes network resources and available bandwidth until the network goes offline.

Analogy: Someone rings every doorbell in an apartment block at once, thousands of times. The doorman runs to each door, finds nobody who asked for him, and writes "no such resident" on a card each time. He never has time to help real visitors.

How it works (numbered steps):

1. The attacker (often through a botnet) sends many UDP packets with spoofed source IPs to random destination ports on the target.
2. For each packet, the server checks whether an application is listening on that port.
3. No application is listening (the ports are non-existent services), so the server repeatedly checks and finds nothing.
4. The server sends back an ICMP "Destination Unreachable" error packet for each one.
5. Work spent on checking and replying, plus the bandwidth used by the flood, makes the legitimate applications inaccessible.

```
Attacker --UDP packet (random port)--> Target server
Attacker --UDP packet (random port)--> Target server
Attacker --UDP packet (random port)--> Target server
      <--ICMP destination unreachable-- (one per packet)
```

Real-world use: An attacker floods a game server's IP with UDP packets to random ports. Players get lag and disconnects. The server's CPU is busy generating ICMP replies, and the uplink is full.

Tools and commands used here: LOIC (UDP mode), hping3 with --udp (section 5), UltraDDOS-v2, ISB.

AI angle: AI can help vary packet size and timing to evade simple rate rules. Defender prompt: "Explain why many ICMP type 3 code 3 messages leaving my server could mean I am under a UDP flood."

Countermeasures and defense: Rate limit UDP, block UDP to ports that should not be open, limit ICMP "Destination Unreachable" replies, use ingress filtering to drop spoofed sources, and use ISP or cloud scrubbing for large floods.

EXAM FOCUS:

- Keywords: "spoofed UDP packets", "random ports", "ICMP Destination Unreachable" -> UDP flood.
- Family: volumetric (bps).
- Trap: UDP flood is not a TCP attack. There is no SYN and no handshake.
- Look-alike: UDP flood (random ports on one target, replies are ICMP errors) vs NTP amplification (UDP to a third-party NTP server with a spoofed victim IP).

Memory trick: "UDP = Unknown Door Pounding." Random doors, error reply each time.

3.8 ICMP Flood Attack [page 4]

What it is (simple words): Internet Control Message Protocol (ICMP) is used by network administrators mainly for IP operations, troubleshooting, and error messages about undeliverable packets. In an ICMP flood, attackers send large volumes of ICMP echo request packets (the packets behind "ping") to a victim, either directly or through reflection networks. The victim answers each one, and the combined traffic saturates the victim's network connection. The victim becomes overwhelmed and stops responding to legitimate TCP/IP requests.

Analogy: Thousands of people call a receptionist and say only "Hello, are you there?" She must answer each call. Real callers get a busy signal.

How it works (numbered steps):

1. The attacker sends ICMP ECHO requests, often with spoofed source addresses.
2. The target replies to each with an ECHO reply, using its own CPU and upload bandwidth.
3. When the request rate is high enough, the link saturates.
4. The target stops responding to legitimate TCP/IP traffic.

The slide also shows the protection: set a threshold limit on ICMP echo requests per second. When the limit is exceeded, the ICMP flood protection feature is invoked and extra requests are dropped. A legitimate ICMP echo request from an address in the same security zone is still allowed through below the threshold.

```
Attacker --ECHO request (spoofed source)--> Target server
        <-------------- ECHO reply ------- Target server
... repeated at very high rate ...
[Maximum limit of ICMP ECHO requests per second]  <- threshold
requests above the limit are dropped
```

Real-world use: A small ISP link of 100 Mbps is hit by ping floods from a botnet. The router's CPU and the link are saturated. The ISP adds an ICMP rate limit, and normal traffic recovers.

Tools and commands used here: ping (flood mode), hping3 with --icmp and --flood, LOIC (ICMP mode), ISB (section 5).

AI angle: Defenders can ask AI to draft a rate-limit policy. Example prompt: "Write a Cisco IOS policer that limits ICMP echo to 100 kbps on the outside interface and explain each line."

Countermeasures and defense: Set a threshold limit for ICMP echo requests per second, rate limit ICMP at the edge, block ICMP from outside where it is not needed, apply ingress filtering, and use ISP scrubbing.

EXAM FOCUS:

- Keywords: "ICMP echo request packets", "reflection networks", "threshold limit" -> ICMP flood and its defense.
- Family: volumetric (bps).
- Trap: do not block all ICMP blindly. Path MTU discovery and troubleshooting need some ICMP types.
- Look-alike: ICMP flood (direct, many echo requests) vs Smurf (echo request sent to a broadcast address with the victim's IP as the spoofed source).

Memory trick: "Ping, ping, ping, ping... until the line breaks."

3.9 Ping of Death Attack [page 5]

What it is (simple words): In a Ping of Death (PoD) attack, the attacker tries to crash, destabilize, or freeze the target system or service by sending malformed or oversized packets with a simple ping command. The IP standard (RFC 791) limits an IP packet to 65,535 bytes. The attacker sends data that adds up to more than that limit, using fragments. The receiving system must reassemble the fragments, and the oversized result can overflow its buffer and crash it.

Analogy: Mailing a 70-page letter in a series of envelopes to a mailbox that holds 65 pages. When the clerk tapes the pages together, the mailbox bursts.

How it works (numbered steps):

1. The attacker crafts an ICMP echo request whose total size exceeds the allowed limit. The slide example: a packet of 65,538 bytes sent to the target web server (the diagram shows 20 bytes IP header + 8 bytes ICMP header + 65,510 bytes ICMP data; the sum is 65,538).
2. The packet is split into fragments to travel the network.
3. The target collects fragments and starts the reassembly process.
4. The reassembled packet is larger than the buffer size allows.
5. The buffer overflows and the system may crash, freeze, or reboot.

```
|<-- 20 bytes -->|<-- 8 bytes -->|<------ 65,510 bytes ------>|
[  IP header  ][ ICMP header ][       ICMP data        ]
 Total = 65,538 bytes  >  limit 65,535 bytes (RFC 791)
```

Real-world use: In the 1990s, one oversized ping crashed many operating systems. Modern systems are patched against it, so it is mostly a historic and exam topic. It can still affect old embedded devices and unpatched legacy systems.

Tools and commands used here: ping with a large size (Windows -l, Linux -s), hping3. Details in section 5.

AI angle: None specific. A defender can ask AI to explain old CVEs or review whether legacy devices on the network are still vulnerable.

Countermeasures and defense: Patch operating systems, drop or block fragments and oversized ICMP at the firewall, and limit ICMP size. Most current stacks check the total size during reassembly and discard oversized packets.

EXAM FOCUS:

- Keywords: "malformed or oversized packets", "simple ping command", "exceeds 65,535 bytes", "RFC 791" -> Ping of Death.
- Family: volumetric on this slide.
- Trap: the crash happens at reassembly on the target, not on the sender or the router.
- Look-alike: Ping of Death (one big malformed packet, crash) vs ICMP flood (many normal pings, saturation).

Memory trick: "Too big to fit: 65,535 is the limit."

3.10 Smurf Attack [page 5]

What it is (simple words): In a Smurf attack, the attacker spoofs the source IP address with the victim's IP address and sends a large number of ICMP ECHO request packets to an IP broadcast network. Every host on that broadcast network replies to the echo request. All the replies go to the victim, whose machine is flooded and may crash.

Analogy: Someone writes "Please reply to Alice" on a thousand postcards and sends them to an apartment block's shared mailbox. Every resident answers, and Alice's door is buried in replies. She never sent anything.

How it works (numbered steps):

1. The attacker finds a network that responds to broadcast pings (the "amplifier" or IP broadcast network).
2. The attacker sends ICMP echo requests to the broadcast address, with the victim's IP as the source.
3. All hosts on the broadcast network respond.
4. The responses go to the victim machine, not the attacker.

5. The victim's link is saturated, and the machine may crash.

```
Attacker -- echo request, source = Victim IP --> Broadcast network
                                    (many hosts)
Victim <============ many echo replies ========== each host replies
```

Real-world use: One 1 Mbps stream from an attacker becomes hundreds of Mbps at the victim if 200 hosts answer. The victim sees attack traffic from the amplifier network, not from the attacker.

Tools and commands used here: hping3 (ICMP with spoofed source and broadcast destination), Smurf scripts. ADDED CONTEXT: the UDP version of this attack is called Fraggle (UDP echo to broadcast).

AI angle: None specific.

Countermeasures and defense: Disable IP directed broadcasts on routers (Cisco: no ip directed-broadcast), configure hosts not to answer ICMP to broadcast addresses, use ingress filtering, and apply rate limits.

EXAM FOCUS:

- Keywords: "spoofed source = victim", "ICMP ECHO to IP broadcast network" -> Smurf.
- Family: volumetric, amplification (the broadcast network multiplies).
- Trap: the replies are the attack. The attacker sends only requests.
- Look-alike: Smurf (ICMP, broadcast) vs Fraggle (UDP, broadcast) vs NTP amplification (UDP, monlist).

Memory trick: "Smurfs live in a village: shout to the whole village and everyone answers the victim."

3.11 NTP Amplification Attack [page 5]

What it is (simple words): Network Time Protocol (NTP) keeps clocks in sync over UDP port 123. In an NTP amplification attack, the attacker uses a botnet to send small UDP requests through a spoofed IP address (the victim's) to NTP servers. The attack is often started through an NTP server that has the monlist command enabled. Each small request triggers a monlist query that produces large response packets, and all responses are sent to the victim.

Analogy: You mail a postcard to a library asking "send me the list of everyone who visited this year", with the victim's address as the return address. The library sends a heavy box to the victim. Your postcard weighed almost nothing.

How it works (numbered steps):

1. The attacker builds a botnet and gives each bot the victim's IP address as the spoofed source.
2. Each bot sends a small UDP request to an NTP server that has monlist enabled.

3. The monlist command returns a list of up to the last 600 hosts that talked to the server (ADDED CONTEXT: verify the exact count).
4. The NTP server sends large response packets to the spoofed address, the victim.
5. Many NTP servers do this at once, and the victim is flooded.

```
Botnet (spoofed source = Victim)
  |       |       |
  v       v       v    small spoofed requests
NTP#1    NTP#2    NTP#3  (monlist enabled)
  \       |       /
   +=======+=======+     large responses
           v
      Victim (original IP)
```

Real-world use: In 2013 to 2014, attackers used open NTP servers to generate some of the biggest floods seen at that time, with an amplification factor of hundreds of times (ADDED CONTEXT, verify the exact figure).

Tools and commands used here: ntpdc -c monlist (query), nmap ntp-monlist script. Details in section 5.

AI angle: Defenders can use AI to scan their asset list and flag servers that still expose NTP monlist. Example prompt: "Write a checklist to find and disable monlist on ntpd servers."

Countermeasures and defense: Disable monlist (use a current ntpd, or add noquery in restrict lines), restrict NTP to trusted clients, filter UDP 123 at the edge where not needed, and apply ingress filtering (BCP 38) to stop spoofing.

EXAM FOCUS:

- Keywords: "monlist", "NTP server", "spoofed IP", "large response" -> NTP amplification.
- Family: volumetric, amplification.
- Trap: the NTP server is not the victim. It is an unwitting reflector.
- Look-alike: NTP amplification vs DNS amplification (same idea, different protocol) vs Smurf (ICMP broadcast).

Memory trick: "Monlist = Money for the attacker: a small coin in, a big bag out."

3.12 SYN Flood Attack [page 6]

What it is (simple words): A SYN flood abuses the TCP three-way handshake. The attacker sends a large number of SYN requests with fake source IP addresses to the target. The target replies with SYN/ACK and waits for the final ACK. Because the source addresses are fake, the ACK never comes. The half-open connections fill the target's "listen queue" until real clients cannot connect.

Analogy: A hotel front desk takes bookings by phone and holds a room for 75 seconds while waiting for a confirmation. A prankster makes thousands of bookings and never confirms. All rooms are "held", and real guests are turned away.

How it works (numbered steps):

1. Normal handshake: Host A sends SYN, Host B replies SYN/ACK, Host A sends ACK, and the connection opens.
2. The attacker sends many SYN packets with fake source IPs to Host B (the victim).
3. Host B sends SYN/ACK to each fake address and waits for the ACK.
4. The fake hosts never answer, so Host B does not get the response.
5. SYN flooding exploits a flaw in how most hosts implement the handshake: Host B must keep track of each partially opened connection in a listen queue for at least 75 seconds.
6. The listen queue is small and fills quickly.
7. The ability to delay each incomplete connection for 75 seconds can be used cumulatively as a DoS attack.

```
Normal:          SYN ->     SYN/ACK <-     ACK ->    (open)
SYN flood:   SYN -> SYN -> SYN -> SYN -> ...   (fake sources)
          <- SYN/ACK sent to nowhere, no ACK ever returns
          Listen queue: [half][half][half][half][FULL]
```

Real-world use: A web server accepts 1,024 half-open connections. An attacker sends 5,000 SYNs per second from random IPs. After a few seconds, the queue is full, and customers see browser timeouts even though bandwidth use is low.

Tools and commands used here: hping3 -S --flood --rand-source (section 5), LOIC (TCP mode), Metasploit synflood module (ADDED CONTEXT), netstat to see SYN_RECV entries.

AI angle: Defenders can ask AI to read netstat or log output and count half-open connections per source. Example prompt: "Here is netstat -an output. How many connections are in SYN_RECV, and do the source IPs look spoofed?"

Countermeasures and defense: TCP Intercept on routers (section 3.25), SYN cookies (ADDED CONTEXT: the server encodes connection state in the sequence number so it needs no queue entry until the real ACK returns), larger backlog queue, shorter SYN timeout, firewalls with SYN proxy, ingress filtering, and rate limiting.

EXAM FOCUS:

- Keywords: "fake source IP", "SYN/ACK", "listen queue", "75 seconds", "never replying" -> SYN flood.
- Family: protocol (pps).
- Trap: bandwidth may look normal. The resource exhausted is the connection queue, not the pipe.
- Look-alike: SYN flood (half-open, no ACK) vs ACK flood (only ACKs, no handshake) vs spoofed session flood (full fake session).

Memory trick: "SYN, SYN, SYN... nobody says ACK."

3.13 Fragmentation Attack [page 6]

What it is (simple words): A fragmentation attack stops a victim from re-assembling fragmented packets. The attacker floods the target with TCP or UDP fragments, which reduces performance. Large fragmented packets (1500+ bytes) are sent to a target web server at a relatively small packet rate. Because the protocol allows fragmentation, the packets usually pass uninspected through routers, firewalls, and IDS/IPS. Reassembling and inspecting them consumes excessive resources. The attacker also randomizes the content of the fragments, which makes reassembly use even more resources and can crash the system.

Analogy: A jigsaw puzzle delivered with pieces from a thousand different puzzles mixed together. The worker has to try every piece, and the table fills up with pieces that never fit.

How it works (numbered steps):

1. The attacker splits a large payload into fragments (original packet -> IP header + data segment 1 to 4 -> fragment 1 to 4).
2. The attacker sends many fragments, often with random content and incomplete sets.
3. Network devices let them pass because fragmentation is legal.
4. The target stores the fragments and tries to reassemble them.
5. Reassembly and inspection consume CPU and memory.
6. Performance drops and the system may crash.

```
Original packet: [IP hdr][seg1][seg2][seg3][seg4]
Fragments sent:  [IP][seg1]  [IP][seg2]  [IP][seg3]  [IP][seg4]
Target must hold all pieces in memory and rebuild the packet
```

Real-world use: A firewall in front of a web server is saturated because it queues fragments waiting for missing pieces. The web server behind it is fine, but the firewall's memory is full.

Tools and commands used here: hping3 --frag, fragroute, Scapy (ADDED CONTEXT; they are also used for IDS evasion in Module 12).

AI angle: None specific.

Countermeasures and defense: Limit reassembly timeouts and buffers, drop fragments that make no sense (tiny or overlapping), use devices that reassemble before inspecting, and apply rate limiting.

EXAM FOCUS:

- Keywords: "re-assemble fragmented packets", "1500+ byte", "pass uninspected" -> fragmentation attack.
- Family: protocol.
- Trap: the packet rate is small. The damage comes from reassembly cost.
- Look-alike: fragmentation attack (flood of fragments) vs Ping of Death (fragments that reassemble to an oversized packet).

Memory trick: "Frag = fragments of fury: easy to send, costly to rebuild."

3.14 Spoofed Session Flood Attack [page 7]

What it is (simple words): Attackers create fake or spoofed TCP sessions by sending multiple SYN, ACK, and RST or FIN packets. They use the attack to bypass firewalls (which only see what look like valid sessions) and perform DDoS attacks against the target network, exhausting its network resources. The slide lists two variants.

Variant	How the fake session is built
Multiple SYN-ACK spoofed session flood	Multiple SYN and ACK packets plus one or more RST or FIN
Multiple ACK spoofed session flood	Skips SYN, uses only ACK packets plus RST or FIN

Analogy: A forger prints fake tickets with the right look. Guards see tickets, think each is a real visit, and let people in. The venue fills up with fakes.

How it works (numbered steps):

1. The attacker sends a mix of SYN, ACK, and RST or FIN packets that look like a valid TCP session.
2. In the SYN-ACK variant, the sequence includes SYN and ACK packets along with one or more RST or FIN packets.
3. In the ACK variant, the attacker completely skips the SYN packets and uses only multiple ACK packets with one or more RST or FIN packets.
4. A stateful firewall may treat the traffic as an established session and let it pass (it has not seen a SYN, or it is simply fooled).
5. The target network spends resources tracking and processing the fake sessions.

Real-world use: Attackers target a company whose firewall only checks for "packets that belong to an established session". The fake ACK flood looks established and goes through, so the firewall's state table fills.

Tools and commands used here: hping3 with -A and -R or -F flags (ADDED CONTEXT).

AI angle: None specific.

Countermeasures and defense: Strict stateful inspection (drop ACKs with no SYN seen), rate limit per source, SYN proxy, and an IPS tuned for TCP anomalies.

EXAM FOCUS:

- Keywords: "fake or spoofed TCP sessions", "bypass firewalls", "RST or FIN" -> spoofed session flood.
- Variant clue: "completely skipping the SYN packets" -> multiple ACK variant.
- Family: protocol.

- Look-alike: spoofed session flood (mimics a full session) vs SYN flood (only half-open handshakes).

Memory trick: "Fake ID at the door: looks like a real session."

3.15 HTTP GET/POST Attack [page 7]

What it is (simple words): HTTP clients such as web browsers connect to a web server through the HTTP protocol to send HTTP requests, which are either GET (ask for a page) or POST (send data). An HTTP GET/POST attack misuses these requests to tie up the web server. It is an application layer attack, and it can use normal-looking requests, so it is hard to tell from real users.

Analogy: A customer starts to give an order and then stops mid sentence. The waiter stands there waiting for the rest. Many such customers, and every waiter is stuck.

How it works (numbered steps):

HTTP GET attack:

1. The attacker uses a time-delayed HTTP header to maintain HTTP connections.
2. The request header is never completed, so the server waits.
3. Many such connections exhaust the web server resources.

HTTP POST attack:

1. The attacker sends HTTP requests with complete headers but with incomplete message bodies.
2. The headers claim a body of a certain length, but the body arrives very slowly or not at all.
3. The server is prompted to wait for the rest of the message body.
4. The server's connection slots stay occupied.

```
HTTP GET  : Attacker -- request, time-delayed header --> Server
            Server waits for the complete header
HTTP POST : Attacker -- full headers, incomplete body --> Server
            Server waits for the message body
```

Real-world use: A login page that accepts POST requests is hit with slow POST bodies from 5,000 bots. Each holds a worker thread. The site runs out of workers, and real users cannot log in, while network bandwidth stays almost empty.

Tools and commands used here: HULK (HTTP flood), HOIC, LOIC (HTTP mode), R-U-Dead-Yet / slow POST tools (ADDED CONTEXT, verify).

AI angle: AI can help generate realistic request headers and human-like timing so a bot looks less like a bot. Defenders use behavioral bot detection. Example prompt (defender): "Rank these URLs by the ratio of requests from sessions that never completed a page load."

Countermeasures and defense: Set short request header and body timeouts, limit connections per IP, use a reverse proxy or load balancer that buffers requests, use a WAF with rate limiting, and use a CDN or DDoS protection service.

EXAM FOCUS:

- Keywords: "time-delayed header" -> GET attack. "complete headers but incomplete message bodies" -> POST attack.
- Family: application layer (rps).
- Trap: the attack uses legal HTTP, so a signature firewall may not see it as malicious.
- Look-alike: HTTP GET/POST (one slow part per request) vs Slowloris (partial requests that are kept alive by trickling headers).

Memory trick: "GET waits for the Header, POST waits for the Package."

3.16 Slowloris Attack [page 7]

What it is (simple words): In a Slowloris attack, the attacker sends partial HTTP requests to the target web server or application. Upon receiving them, the server opens multiple connections and keeps waiting for the requests to complete. The requests never complete, so the server's maximum concurrent connection pool is exhausted and additional connection attempts are denied. The attacker needs very little bandwidth.

Analogy: A bank has 50 counters. The attacker sends 50 people, each of whom starts a transaction and says "wait, I need to find my ID" every few seconds forever. All counters stay busy, and nobody else gets served.

How it works (numbered steps):

1. The attacker opens many HTTP connections to the web server.
2. On each, the attacker sends a partial HTTP request.
3. The attacker adds a small extra header line every few seconds to keep each connection alive (ADDED CONTEXT).
4. The server keeps each connection open, waiting for the request to finish.
5. The connection pool fills up.
6. New, real connection attempts are denied.

```
Normal : request ----------------------> server
       <--------------- HTTP response
Slowloris: partial request ----(waits)----> server
         partial request ----(waits)----> server
         partial request ----(waits)----> server  (pool full)
```

Real-world use: One laptop running a Slowloris script takes down an Apache server with default settings, because Apache gives each connection a worker. The server's CPU is idle, but no worker is left.

Tools and commands used here: Slowloris (the tool of the same name), slowhttptest (ADDED CONTEXT).

AI angle: None specific beyond general script help.

Countermeasures and defense: Limit connections per IP, set request timeouts, use event-based servers such as nginx (ADDED CONTEXT), put a reverse proxy in front, and use mod_reqtimeout or similar controls on Apache (verify).

EXAM FOCUS:

- Keywords: "partial HTTP requests", "maximum concurrent connection pool exhausted" -> Slowloris.
- Family: application layer.
- Trap: Slowloris is low-bandwidth. A question that says "traffic volume is normal but the server is unreachable" points here.
- Look-alike: Slowloris (many half-sent requests) vs HTTP flood (many complete requests).

Memory trick: "Slow loris = slow animal: one drop at a time, blocks the whole road."

3.17 Multi-Vector Attack [page 8]

What it is (simple words): In a multi-vector DDoS attack, attackers use a combination of volumetric, protocol, and application-layer attacks to disable the target. They rapidly and repeatedly change the form of the DDoS attack (for example, SYN packets, then Layer 7). The attacks are either launched one vector at a time or in parallel, to confuse the company's IT department and exhaust its resources while its focus is diverted to the wrong solution.

Analogy: A burglar sets off the front door alarm, then the back door alarm, then the window alarm. The guard runs from one to the other and never finds the real break-in.

How it works (numbered steps):

1. The attacker starts with one vector (for example, a volumetric UDP flood).
2. The defender responds with a fix for that vector.
3. The attacker switches to another vector (a SYN flood) or adds it at the same time.
4. The defender's tools and attention are spread across several fronts.
5. The attacker keeps rotating until something breaks.

```
IN SEQUENCE : Volumetric -> Protocol -> Application layer -> Victim
IN PARALLEL : Volumetric  ====+
              Protocol    ====+====> Victim
              Application ====+
```

Real-world use: A gaming company mitigates a UDP flood within minutes. Immediately a layer 7 flood hits the login API. The team thinks the first fix failed, loses an hour, and the outage grows.

Tools and commands used here: Any combination of earlier tools. Commercial booter and stresser services bundle several vectors.

AI angle: AI can automate the switching: it watches defender responses and picks a new vector. Defenders use AI correlation to see that the vectors share one botnet. Example prompt (defender): "Combine these three alerts and say if they share source ASNs or timing."

Countermeasures and defense: Layered defense: capacity plus scrubbing for volumetric, TCP Intercept and SYN cookies for protocol, WAF and rate limiting for application layer, and a tested DDoS runbook.

EXAM FOCUS:

- Keywords: "combination of volumetric, protocol, and application layer", "change the form", "diverted to the wrong solution" -> multi-vector.
- Trap: sequence and parallel are both multi-vector.
- Best-answer rule: defend all layers at once, not just the loudest one.

Memory trick: "Many hands: V-P-A together."

3.18 Permanent Denial-of-Service Attack [page 8]

What it is (simple words): A permanent DoS (PDoS) attack, also known as phlashing, causes irreversible damage to system hardware. Unlike other DoS attacks, it sabotages the hardware itself, so the victim must replace or reinstall the hardware. The attack is carried out using a method known as "bricking a system", which means the device becomes as useless as a brick. Attackers do this by sending fraudulent hardware updates (fake firmware) to the victim.

Analogy: Most DoS attacks are like blocking a road. PDoS is like blowing up the bridge. Clearing the traffic does not help.

How it works (numbered steps):

1. The attacker sends email, IRC chats, tweets, posts, or videos that carry fraudulent content for a hardware update.
2. The victim is tricked into running the "update".
3. The attacker gets access to the victim's computer when the malicious code is executed.
4. The fake firmware overwrites the real firmware of a device such as a router, BIOS, printer, or camera.
5. The device is bricked. It will not boot or work, and the victim must replace or reinstall the hardware.

```
Attacker -> email/chat/tweet with fake hardware update -> Victim
Victim runs update -> malicious code executes -> firmware overwritten
Result: bricked device (replace or reinstall hardware)
```

Real-world use: Researchers have shown that unpatched routers and IoT devices exposed to the Internet can be flashed with bad firmware. The BrickerBot malware (ADDED CONTEXT, around 2017) destroyed poorly secured IoT devices this way.

Tools and commands used here: None on the slide. ADDED CONTEXT: BrickerBot is the well-known example; it is not named on the slide.

AI angle: AI can write convincing "firmware update" phishing messages. Defenders can use AI to check firmware vendor notices against official sources. Example prompt: "Does this email look like a genuine vendor firmware notice? List the red flags."

Countermeasures and defense: Download firmware only from official vendor sites, verify hashes and digital signatures, use secure boot, keep backups of known-good firmware, change default passwords, do not expose management interfaces to the Internet.

EXAM FOCUS:

- Keywords: "phlashing", "bricking", "irreversible damage to hardware", "fraudulent hardware updates" -> permanent DoS.
- Trap: the recovery is replacement or reinstall of hardware, not waiting for the attack to stop.
- Look-alike: PDoS (hardware damage) vs normal DoS (temporary, stops when traffic stops).

Memory trick: "Phlash = Flash gone bad. Bricked for good."

3.19 TCP SACK Panic Attack [pages 8 and 9]

What it is (simple words): In a TCP SACK Panic attack, the attacker tries to crash a target Linux machine by sending Selective Acknowledgement (SACK) packets with a malformed Maximum Segment Size (MSS). SACK is a TCP option that lets a receiver say exactly which pieces of data it got. The attack exploits an integer overflow vulnerability in the Linux Socket Buffer (SKB), which can lead to a kernel panic (a fatal operating system crash). Page 9 repeats the same slide as page 8.

Analogy: A bank clerk keeps a counter of parcels on a small mechanical tally. The attacker sends so many tiny parcels in a special order that the counter rolls over from its maximum back to zero. The clerk's whole system jams.

How it works (numbered steps):

1. The attacker sends SACK packets in sequence to the target Linux server.
2. The attacker sets the MSS to the lowest value (48 bytes on the slide), which forces the data into very many tiny segments.
3. The socket buffer exceeds its limit.
4. An integer overflow is triggered in the socket buffer (SKB) code.
5. The kernel panics, the server crashes, and legitimate users lose service.

```
Attacker --SACK packet (MSS = 48)--> Linux server
Attacker --SACK packet (MSS = 48)--> Linux server
Socket buffer limit exceeded -> integer overflow -> KERNEL PANIC
Legitimate users -X-> cannot connect
```

Real-world use: In June 2019, researchers disclosed this flaw in Linux kernels (CVE-2019-11477, nicknamed "SACK Panic"; ADDED CONTEXT). A single remote attacker could crash an unpatched server with a stream of crafted packets. Vendors shipped kernel patches within days.

Tools and commands used here: Crafted packets (hping3, Scapy). Mitigation commands in section 5.

AI angle: Defenders can use AI to read vulnerability advisories and match them to the kernel versions in their asset list. Example prompt: "Which of these kernel versions are affected by CVE-2019-11477 and what is the fixed version?" (Always verify against the vendor advisory.)

Countermeasures and defense: Patch the Linux kernel, or as a workaround disable SACK (net.ipv4.tcp_sack=0), or block connections with very low MSS using a firewall rule (ADDED CONTEXT, verify the exact rule for your distribution). Disabling SACK can reduce performance.

EXAM FOCUS:

- Keywords: "SACK", "MSS", "integer overflow", "SKB", "kernel panic", "Linux" -> TCP SACK Panic.
- Family: protocol attack on this module's slide.
- Number: MSS set to 48 bytes (lowest value, from the slide).
- Trap: it only affects Linux kernels with the flaw, not Windows.

Memory trick: "SACK + low MSS = kernel panic."

3.20 Distributed Reflection Denial-of-Service (DRDoS) Attack [page 9]

What it is (simple words): A DRDoS attack, also known as a spoofed attack, uses multiple intermediary and secondary machines that contribute to the real DDoS attack against the target. The attacker sends requests to the intermediary hosts, which redirect the requests to the secondary machines. The secondary machines then reflect the attack traffic to the target.

Analogy: Instead of throwing stones yourself, you ask a hundred neighbors to throw stones for you, each thinking they are answering a question from the victim. The victim sees neighbors throwing stones, not you.

How it works (numbered steps):

1. The attacker sends requests with the victim's IP as the source to intermediary victims.
2. The intermediary hosts pass or redirect those requests to secondary victims (reflectors).

3. The secondary victims send their replies to the primary target.
4. The target is hit by traffic from many reflectors.

Advantages for the attacker (from the slide):

- The primary target appears to be directly attacked by the secondary victim rather than by the actual attacker. This hides the attacker.
- Multiple intermediary victim servers are used, which increases the attack bandwidth.

```
Attacker -> Intermediary victims -> Secondary victims -> Target
(target sees the secondary victims as the source, not the attacker)
```

Real-world use: Open DNS resolvers, NTP servers, and memcached servers have been used as reflectors (ADDED CONTEXT). The victim traces the traffic back to those innocent servers.

Tools and commands used here: None named on the slide.

AI angle: None specific.

Countermeasures and defense: Ingress filtering (BCP 38) at ISPs stops spoofed sources, reflectors should not be open to the Internet, and the target uses rate limiting and scrubbing.

EXAM FOCUS:

- Keywords: "intermediary and secondary", "reflect", "spoofed attack" -> DRDoS.
- Trap: a DDoS has bots sending directly. A DRDoS has innocent reflectors sending.
- Look-alike: DRDoS vs DDoS vs amplification (reflection plus a response bigger than the request).

Memory trick: "Reflection = mirror: the attacker hides behind it."

3.21 DoS/DDoS Attack Toolkits in the Wild [page 10]

What it is (simple words): Attack toolkits are programs that make DoS and DDoS easy, often with a point-and-click interface. Anyone can download them. The slide names seven tools. URLs on the slide were too small to read [UNCLEAR on page 10].

Tool	What the slide says or what it is
ISB (I'm so bored)	Utility for HTTP, UDP, TCP, and ICMP floods
UltraDDOS-v2	GUI: target IP, port, number of packets
High Orbit Ion Cannon (HOIC)	Flood tool (HTTP floods, ADDED CONTEXT)
Low Orbit Ion Cannon (LOIC)	Flood tool (TCP, UDP, HTTP, ADDED CONTEXT)
HULK	HTTP Unbearable Load King (ADDED CONTEXT)
Slowloris	Holds connections open with partial requests

Tool	What the slide says or what it is
UFONet	Botnet-style DoS using web vulnerabilities (ADDED CONTEXT)

Analogy: A set of power tools. Anyone who can press a button can run one, but the damage can be large.

How it works (numbered steps):

1. The attacker enters the target IP or URL and, for some tools, a port and the number of packets.
2. The attacker picks the protocol (HTTP, UDP, TCP, ICMP).
3. The tool floods the target with requests.
4. Attackers with many machines run the tool on all of them, or use volunteers (LOIC was used in voluntary "hacktivist" campaigns, ADDED CONTEXT).

Tool notes:

- ISB: "I'm so bored". A software utility that lets attackers perform HTTP, UDP, TCP, and ICMP flood attacks on the target network.
- UltraDDOS-v2: provides a simple GUI where attackers enter the target IP address, port number, and the number of packets they want to send. The screenshot shows a Wireshark-like packet list of the generated traffic.
- LOIC: simple GUI flooder. It does not hide the user's IP by default, so users are easy to trace (ADDED CONTEXT).
- HOIC: successor to LOIC aimed at HTTP floods using "booster" scripts to vary requests (ADDED CONTEXT).
- HULK: generates unique, obfuscated HTTP requests to bypass caches (ADDED CONTEXT).
- Slowloris: low-bandwidth tool. See section 3.16.
- UFONet: ADDED CONTEXT. It uses open redirect or similar flaws on third-party websites to make those sites send requests at the target.

Real-world use: A pen tester with written permission runs a flood tool against a staging server to measure how much load it survives. The result goes into the report as a capacity and resilience finding.

Tools and commands used here: All of the above, plus hping3 (section 5).

AI angle: Attackers can ask AI to write custom flooders. Public AI models have safeguards and will refuse many such requests, but criminal AI tools such as WormGPT and FraudGPT-style services (ADDED CONTEXT) are marketed for malicious scripting. Defenders can use AI to write detection rules. Example prompt (defender): "Write a Suricata rule that alerts on more than 100 SYN packets per second from one source."

Countermeasures and defense: Rate limiting, WAF rules for known tool signatures (LOIC and HOIC leave recognizable patterns), IP reputation filtering, and DDoS protection services.

EXAM FOCUS:

- Keywords: "GUI", "target IP, port number, number of packets" -> UltraDDOS-v2. "I'm so bored" -> ISB.
- "Ion Cannon" -> LOIC or HOIC. LOIC = Low, HOIC = High.
- "HTTP Unbearable Load King" -> HULK.
- "Partial HTTP requests" -> Slowloris.
- Trap: these are attack tools, not defense tools.

Memory trick: "Ion cannons shoot, HULK smashes, Slowloris sneaks, UFO nets reach out."

3.22 Detection Techniques [page 10]

What it is (simple words): Detection techniques are based on identifying and discriminating illegitimate traffic increases and flash events from legitimate packet traffic. A flash event is a sudden but legitimate surge (for example, a sale or breaking news). All detection techniques define an attack as an abnormal and noticeable deviation from a threshold of normal network traffic statistics. The slide lists three techniques.

Technique	Core idea
Activity profiling	Compare average packet rate of flows to a profile
Sequential change-point detection	Spot a change in traffic statistics (Cusum)
Wavelet-based signal analysis	Examine traffic as signals in spectral parts

Analogy: A nurse watches a heart monitor. Activity profiling is "the usual pulse is 70 and now it is 140". Change-point detection is "the line suddenly changed shape at 3:02 pm". Wavelet analysis is "split the signal into frequencies and look for odd ones".

How it works (numbered steps):

Activity profiling:

1. It is based on the average packet rate for a network flow, which consists of consecutive packets with similar packet fields.
2. Activity profiles are obtained by monitoring network packet header information.
3. An attack is indicated by an increase in activity levels among the network flow clusters, and by an increase in the overall number of distinct clusters (a DDoS attack).

Sequential change-point detection:

1. Change-point detection algorithms isolate changes in network traffic statistics and in traffic flow rate caused by attacks.
2. The algorithm filters the target traffic data by address, port, or protocol and stores the resultant flow as a time series.
3. The technique uses the Cusum (Cumulative Sum) algorithm to identify and locate DoS attacks.

4. It can also identify the typical scanning activities of network worms.

Wavelet-based signal analysis:

1. Wavelet analysis describes an input signal in terms of spectral components.
2. Analyzing each spectral window's energy determines the presence of anomalies.
3. It filters out the anomalous traffic flow input signals from the background noise.

Real-world use: A network monitor computes the packet rate per destination every 10 seconds. When the cumulative sum of the deviation from normal passes a limit, it raises an alert: "possible DDoS toward 203.0.113.10".

Tools and commands used here: NetFlow/sFlow collectors, IDS/IPS, Wireshark, netstat (ADDED CONTEXT). The three techniques are methods, not products.

AI angle: Machine learning is the modern form of these methods. Models learn "normal" and flag deviations. Example prompt (defender): "Explain how a Cusum detector would work on my requests-per-second series and what threshold I should start with."

Countermeasures and defense: Detection is the first step. After an alert, apply the response strategies in section 3.23.

EXAM FOCUS:

- Keywords: "Cusum" -> sequential change-point detection.
- Keywords: "spectral components", "background noise" -> wavelet.
- Keywords: "average packet rate", "network flow clusters" -> activity profiling.
- Trap: a flash event is legitimate. The detection goal is to tell flash events from attacks.
- Count: three detection techniques.

Memory trick: "A-S-W: Activity, Sequential, Wavelet." Cusum = "C for Change".

3.23 DoS/DDoS Countermeasure Strategies [page 11]

What it is (simple words): Three high-level strategies for dealing with an attack in progress.

Strategy	What it means
Absorbing the attack	Use additional capacity to absorb the attack
Degrading services	Keep critical services, stop non-critical ones
Shutting down the services	Shut down all services until the attack stops

Analogy: A flood hits a town. Absorb: build bigger drains ahead of time. Degrade: sandbag only the hospital and power plant and let the rest flood. Shut down: evacuate and switch everything off until the water drops.

How it works (numbered steps):

1. Absorbing: use additional capacity (bandwidth, servers, cloud scrubbing) to take the attack. This requires preplanning and additional resources.
2. Degrading services: identify the critical services and maintain their functionality while stopping non-critical services.
3. Shutting down: shut down all services until the attack has subsided. This is the last resort, because it finishes the attacker's job for them.

Real-world use: An online store under attack turns off its recommendation engine and image galleries (degrade) so checkout stays up. If the attack grows, it pushes traffic to a cloud scrubbing provider (absorb).

Tools and commands used here: Load balancers, auto-scaling, CDN, DDoS protection services (section 3.30).

AI angle: Defenders can use AI in the playbook to rank which services are critical from dependency maps. Example prompt: "From this service dependency list, which services must stay up for payments to work?"

Countermeasures and defense: This slide is the strategy. Specific controls follow in sections 3.24 to 3.30.

EXAM FOCUS:

- Keywords: "additional capacity" -> absorb. "critical services" -> degrade. "until the attack has subsided" -> shut down.
- Best-answer rule: shutting down is the last resort. Prefer absorb or degrade first.
- Count: three strategies.

Memory trick: "A-D-S: Absorb, Degrade, Shut."

3.24 DDoS Attack Countermeasures [page 11]

What it is (simple words): A group of defenses aimed at the DDoS ecosystem itself. The slide has two columns: protect secondary victims (the machines that get turned into bots) and detect and neutralize handlers (the machines that control the bots).

Analogy: To stop a gang, you protect people from being recruited, and you arrest the few bosses who give the orders.

How it works (numbered steps):

Protect secondary victims:

1. Monitor security regularly to remain protected from DDoS agent software.
2. Install anti-virus and anti-Trojan software and keep it up to date.
3. Increase awareness of security issues and prevention techniques among all Internet users.

4. Disable unnecessary services, uninstall unused applications, and scan all files received from external sources.
5. Properly configure and regularly update the built-in defensive mechanisms in the core hardware and software of systems.

Detect and neutralize handlers:

1. Network traffic analysis: analyze communication protocols and traffic patterns between handlers and clients, or between handlers and agents, to identify the network nodes that might be infected by the handlers.
2. Neutralize botnet handlers: there are usually fewer DDoS handlers than agents. Neutralizing a few handlers can render multiple agents useless and thwart DDoS attacks.
3. Spoofed source address: there is a decent probability that the spoofed source address of DDoS attack packets will not represent a valid source address of the definite sub-network, so it can be filtered.

Real-world use: A company blocks the C&C domain used by a botnet. Thousands of infected machines lose their orders at once. This is more effective than blocking each bot.

Tools and commands used here: Anti-virus, EDR, NetFlow analysis, DNS sinkholes (ADDED CONTEXT).

AI angle: AI helps spot beaconing (regular check-ins to a C&C server). Example prompt (defender): "Find hosts whose outbound connections to one domain repeat at fixed intervals of about 60 seconds."

Countermeasures and defense: This is the countermeasure slide. Key point: attack the control layer (handlers, C&C) and the recruitment layer (secondary victims).

EXAM FOCUS:

- Keywords: "handlers" -> neutralize handlers to disable many agents.
- Keywords: "secondary victims" -> the compromised hosts used as bots.
- Trap: "primary victim" is the final target. "Secondary victims" are the bots.
- Best-answer rule: attacking the handler gives more effect per action than cleaning single agents.

Memory trick: "Cut the head, not each finger: few handlers, many agents."

3.25 DDoS Attack Countermeasures: Prevent Potential Attacks [page 12]

What it is (simple words): Four preventive controls: egress filtering, ingress filtering, TCP Intercept, and rate limiting.

Control	Main job
Egress filtering	Scans headers of IP packets leaving a network

Control	Main job
Ingress filtering	Prevents source address spoofing of Internet traffic
TCP Intercept	Protects TCP servers from SYN flood on routers
Rate limiting	Controls rate of traffic on an interface

Analogy: A building has four checks: guards at the exit (egress) stop staff from carrying bad items out, guards at the entrance (ingress) turn away visitors with fake badges, a receptionist (TCP Intercept) answers the door for the office until the visitor proves real, and a turnstile (rate limiting) allows only a few people per minute.

How it works (numbered steps):

Egress filtering:

1. It scans the headers of IP packets leaving a network.
2. It ensures that unauthorized or malicious traffic never leaves the internal network.
3. Packets that do not meet the necessary specifications do not reach the targeted address. A common rule: only allow packets whose source address belongs to your own network.

Ingress filtering:

1. It prevents source address spoofing of Internet traffic.
2. It protects against flooding attacks that originate from valid prefixes (IP addresses).
3. It allows the originator to be traced to its true source.

TCP Intercept:

1. TCP Intercept features in routers protect TCP servers from TCP SYN flooding attacks.
2. Configuring TCP Intercept prevents DoS attacks by intercepting and validating TCP connection requests.
3. ADDED CONTEXT: in intercept mode, the router answers the client's SYN itself, then connects to the server only if the client completes the handshake. In watch mode, the router passively watches and resets connections that do not complete in time.

Rate limiting:

1. It controls the rate of outbound or inbound traffic of a network interface controller.
2. It reduces the high-volume inbound traffic caused by DDoS attacks.

```
Inside network --[egress filter]--> Internet --[ingress filter]--> Inside
Egress : only your own source IPs may LEAVE
Ingress: block spoofed or invalid source IPs ENTERING
```

Real-world use: An ISP customer router drops any outgoing packet whose source is not in the customer's own address range. If one of the customer's PCs becomes a bot, it cannot send spoofed-source floods.

Tools and commands used here: Router ACLs, uRPF, Cisco TCP Intercept, QoS policers (section 5).

AI angle: Defenders can ask AI to turn plain-language policy into router syntax and explain each line. Always test before applying. Example prompt: "Write an IOS ACL that permits only 198.51.100.0/24 as a source on the outbound interface and logs everything else."

Countermeasures and defense: These are the countermeasures. Layer them with the controls in sections 3.26 to 3.30.

EXAM FOCUS:

- Keywords: "headers of IP packets leaving a network" -> egress.
- Keywords: "prevents source address spoofing", "traced to its true source" -> ingress.
- Keywords: "intercepting and validating TCP connection requests" -> TCP Intercept.
- Look-alike: ingress (traffic entering) vs egress (traffic leaving).
- Trap: egress filtering protects OTHERS from your compromised hosts.

Memory trick: "In = Intruders blocked. Out = Our hosts restrained."

3.26 DDoS Attack Countermeasures: Deflect Attacks [page 12]

What it is (simple words): Deflecting means steering attack traffic away from real systems. The slide uses honeypots: systems set up with limited security that act as an enticement for an attacker. Honeypots serve as a means of gaining information about attackers, their attack techniques, and tools by storing a record of the system activities. The defense-in-depth approach uses Intrusion Prevention Systems (IPS) at different network points to divert suspicious DoS traffic to several honeypots. The slide shows Blumira Honeypot software, which helps security professionals detect unauthorized access attempts and the attackers' lateral movement (https://www.blumira.com).

Analogy: A decoy shop next to the real shop. Thieves go to the decoy and the owner watches how they work.

How it works (numbered steps):

1. The defender deploys honeypots that look like real servers but hold no real data.
2. IPS devices placed at several network points spot suspicious DoS traffic.
3. The IPS diverts that traffic to the honeypots.
4. The honeypots log the attacker's tools and techniques.
5. The defender uses the logs to improve filters and share threat intelligence.

Real-world use: A company places a honeypot that looks like an old web server. Scans and floods hit it first. The security team reads the logs and blocks the source ranges at the edge.

Tools and commands used here: Blumira Honeypot. ADDED CONTEXT: other honeypots include Honeyd, Kippo/Cowrie, and T-Pot (Module 12 covers honeypots in detail).

AI angle: AI can summarize honeypot logs into attacker profiles. Example prompt: "Summarize these 5,000 honeypot log lines by source country, tool user agent, and top targeted ports."

Countermeasures and defense: Honeypots are a detection and deflection tool. They do not replace real filtering or scrubbing.

EXAM FOCUS:

- Keywords: "limited security", "enticement", "gaining information about attackers" -> honeypot.
- Keywords: "Blumira" -> honeypot software.
- Trap: honeypots do not stop a volumetric flood by themselves.

Memory trick: "Honey attracts bees: honeypot attracts attackers."

3.27 Techniques to Defend against Botnets [page 13]

What it is (simple words): Four techniques that limit what a botnet can do: RFC 3704 filtering, Cisco IPS source IP reputation filtering, black hole filtering, and DDoS prevention offerings from the ISP or a DDoS service.

Technique	Main idea
RFC 3704 filtering	Deny traffic with spoofed addresses
Cisco IPS source IP reputation filtering	Block IPs known as threat sources
Black hole filtering	Drop packets at the routing level
ISP or DDoS service offerings	Enable IP Source Guard and similar

Analogy: RFC 3704: checks ID. Reputation: a list of known troublemakers at the door. Black hole: a trash chute that swallows traffic. ISP offerings: the landlord's security.

How it works (numbered steps):

RFC 3704 filtering:

1. It limits the impact of DDoS attacks by denying traffic with spoofed addresses.
2. Any traffic coming from unused or reserved IP addresses is bogus and should be filtered at the ISP before it enters the Internet link.

Cisco IPS source IP reputation filtering:

1. Reputation services help determine whether an IP or service is a source of threat.
2. Cisco IPS regularly updates its database with known threats such as botnets, botnet harvesters, and malware, and helps in filtering DoS traffic.

Black hole filtering:

1. A "black hole" is a network node where incoming traffic is discarded or dropped without informing the source that the data did not reach its intended recipient.
2. Black hole filtering means discarding packets at the routing level.

DDoS prevention offerings from the ISP or a DDoS service:

1. Enable IP Source Guard (in Cisco) or similar features in other routers to filter traffic based on the DHCP snooping binding database or IP source bindings.
2. This prevents a bot from succeeding with spoofed packets.

Real-world use: A victim under a 40 Gbps flood asks the ISP to black hole the one attacked IP. The attack stops hurting the rest of the network, but that single IP is unreachable (the attacker "won" for that address). The company then moves the service to a new IP behind a scrubbing provider.

Tools and commands used here: uRPF, ACLs, Cisco IPS, IP Source Guard, remotely triggered black hole (RTBH; ADDED CONTEXT). Commands in section 5.

AI angle: AI can feed reputation systems with fresh indicators. Example prompt: "Merge these three IP blocklists, remove duplicates, and flag any address in my own ranges."

Countermeasures and defense: All four items are countermeasures.

EXAM FOCUS:

- Keywords: "RFC 3704" -> ingress filtering for spoofed addresses (ADDED CONTEXT: RFC 3704 is BCP 84, updating RFC 2827 / BCP 38).
- Keywords: "discarded without informing the source" -> black hole.
- Keywords: "DHCP snooping binding database" -> IP Source Guard.
- Trap: black holing stops the attack traffic but also drops the real traffic to that IP.

Memory trick: "R-C-B-I: Real Cops Block Intruders (RFC, Cisco reputation, Black hole, IP Source Guard)."

3.28 DoS/DDoS Protection at ISP Level [page 13]

What it is (simple words): Large attacks must be stopped upstream, before they fill the victim's own link. The slide lists four points.

1. Most ISPs simply block all requests during a DDoS attack, denying even the legitimate traffic from accessing the service.
2. ISPs offer in-the-cloud DDoS protection for Internet links so that they do not become saturated by the attack.
3. In-the-cloud DDoS protection redirects attack traffic to the ISP during the attack and sends it back (cleaned).

4. Administrators can request ISPs to block the original affected IP and move their site to another IP after performing DNS propagation.

Analogy: A town's water authority diverts a flood to a reservoir upstream, treats it, and sends only clean water to your house.

How it works (numbered steps):

1. The botnet sends traffic toward the target (the diagram shows thousands of bots, each sending a small amount, adding up to many gigabits per second) [UNCLEAR on page 13: exact bot counts and rates].
2. The traffic crosses the Internet backbone, where links of about 10 GB are shown [UNCLEAR on page 13].
3. At the ISP provider network, attack traffic is redirected to a scrubbing center.
4. Clean traffic of about 1 GB is sent back to the target network and the target web server [UNCLEAR on page 13].
5. If needed, the admin asks the ISP to block the affected IP and moves the site to a new IP after DNS propagation.

Real-world use: A company with a 1 Gbps link is hit by 30 Gbps. Its own firewall cannot help because the link is full. The ISP's scrubbing service takes the traffic, removes attack packets, and passes the clean remainder.

Tools and commands used here: ISP scrubbing, BGP diversion, CDN (ADDED CONTEXT).

AI angle: Scrubbing centers use machine learning to tell bot traffic from real users. Example prompt (defender): "Write an incident checklist for calling the ISP during a DDoS."

Countermeasures and defense: In-the-cloud scrubbing, DNS change with low TTL, and an agreed escalation contact at the ISP.

EXAM FOCUS:

- Keywords: "in-the-cloud", "redirects attack traffic to the ISP" -> cloud scrubbing.
- Trap: "block all requests" harms legitimate users too.
- Best-answer rule: when the attack fills the link, only an upstream fix (ISP or cloud) works. A local firewall cannot.

Memory trick: "Wash the traffic upstream."

3.29 DoS/DDoS Protection Tools [page 14]

What it is (simple words): Software and appliances that detect and block DoS and DDoS traffic. The slide names six. URLs were small [UNCLEAR on page 14].

Tool	What it does
Anti DDoS Guardian	Protects IIS, Apache, game, Camfrog, mail, FTP, VOIP PBX, SIP servers

Tool	What it does
DDoS-Guard	DDoS protection (name read from the slide)
DOSarrest DDoS protection service	Cloud DDoS protection
Radware DefensePro X	Hardware/software attack mitigation
Gatekeeper	Open-source DDoS protection (github.com)
F5 DDoS Attack Protection	Protection from F5 (f5.com)

Analogy: Different security companies: some install a guard in your building (software on the server), some send a team to stop crowds before they reach your street (cloud).

How it works (numbered steps):

1. Install or route traffic through the tool.
2. The tool learns normal traffic or uses signatures and limits.
3. When traffic exceeds limits or matches attack patterns, the tool drops or challenges it.
4. Legitimate users keep access.

Real-world use: A small game host installs Anti DDoS Guardian on its Windows server to limit connections per IP and block flood sources.

Tools and commands used here: The six tools above. ADDED CONTEXT: other well-known tools include Cisco Guard, Arbor/NETSCOUT Sightline, and Corero.

AI angle: Most modern products add machine-learning baselining. Example prompt (defender): "Compare two DDoS protection products on detection method, time to mitigate, and cost model."

Countermeasures and defense: These are protection products.

EXAM FOCUS:

- Keywords: "IIS servers, Apache servers, game servers, Camfrog servers, mail servers" -> Anti DDoS Guardian.
- Keywords: "Radware DefensePro" or "F5" -> commercial mitigation.
- Trap: do not confuse protection tools (Gatekeeper, DOSarrest) with attack tools (LOIC, HOIC).

Memory trick: "Guardian guards the server. Gatekeeper keeps the gate."

3.30 DoS/DDoS Protection Services [page 14]

What it is (simple words): Managed cloud services that absorb and clean DDoS traffic for you. The slide features two main services and four additional ones.

Service	Detail from the slide
Cloudflare	Defends networks against DDoS, 100 Tbps infrastructure
Akamai DDoS Protection	Dedicated infrastructure for Internet-facing apps and DNS
Stormwall PRO	Additional service (stormwall.network)
Imperva DDoS Protection	Additional service (imperva.com)
Nexusguard	Additional service (nexusguard.com)
BlockDoS	Additional service (blockdos.com)

The Cloudflare figure "100 Tbps" was read from a small slide; verify the current number with Cloudflare.

Analogy: A bouncer company with a huge staff that stands outside your venue and checks every guest.

How it works (numbered steps):

1. Change DNS or routing so your traffic goes through the provider.
2. The provider's global network absorbs the traffic.
3. Attack traffic is filtered at the edge.
4. Clean traffic is passed to your origin server.

Real-world use: An online retailer places Cloudflare in front of its site. During a flood, the retailer's origin server sees only normal traffic, and customers keep shopping.

Tools and commands used here: DNS and BGP configuration. Provider dashboards.

AI angle: Providers use large-scale AI models trained on traffic from all their customers to spot new attack patterns quickly.

Countermeasures and defense: Also hide the origin server IP, so attackers cannot bypass the service by hitting the origin directly (ADDED CONTEXT).

EXAM FOCUS:

- Keywords: "100 Tbps" -> Cloudflare. "Dedicated infrastructure" and "DNS services" -> Akamai.
- Trap: a cloud service helps only if the origin IP is not exposed.

Memory trick: "Cloudflare = Cloud wall. Akamai = Always-on."

3.31 Module Summary, Botnet Ecosystem, and the Google Cloud HTTP/2 Rapid Reset Case Study [page 15]

What it is (simple words): The summary slide lists what the module covers: concepts of DoS and DDoS attacks, the concept of botnets along with the botnet ecosystem, various types of DoS/DDoS attacks, DoS/DDoS attack tools, a detailed DDoS case study (the DDoS attack on Google Cloud, HTTP/2 "Rapid Reset" attack), and countermeasures with hardware and software protection tools. The next module is Session Hijacking. No screenshots of the botnet ecosystem or the case study were supplied, so the details below are all ADDED CONTEXT.

> **ADDED CONTEXT:** Botnet ecosystem. A botmaster (attacker) controls a C&C server. The C&C talks to bots using IRC, HTTP, or peer-to-peer channels. Bots are infected computers, phones, or IoT devices. The botnet is rented to customers as "DDoS-for-hire" (booter or stresser services). Famous example: Mirai (2016), which used default passwords on IoT cameras and routers.
>
> **ADDED CONTEXT:** HTTP/2 Rapid Reset (CVE-2023-44487). HTTP/2 lets a client open many streams on one connection and cancel a stream with a RST_STREAM frame. The attacker opens a stream, cancels it at once, and repeats this many times. The server still does the work for each request, but the client never waits, so the attacker needs few resources. Google reported a peak above 398 million requests per second in August 2023 and disclosed it on 10 October 2023 (Google Cloud blog). Defense: patch HTTP/2 servers and proxies, limit stream resets per connection, and use a DDoS service.

Analogy: Rapid Reset is a customer who orders and cancels a meal instantly, again and again, so the kitchen starts cooking each order but the customer never queues.

How it works (numbered steps): See the box above. Open stream, send request, send RST_STREAM, repeat on the same connection.

Real-world use: Google, Cloudflare, and Amazon (AWS) reported the attack at the same time and coordinated the disclosure.

Tools and commands used here: None in the notes.

AI angle: None specific.

Countermeasures and defense: Patch, limit resets per connection, and use upstream protection.

EXAM FOCUS:

- Keywords: "HTTP/2", "RST_STREAM", "Rapid Reset", "Google Cloud" -> application layer DDoS (layer 7).
- Trap: it is a layer 7 attack measured in rps, not a volumetric bps attack.

Memory trick: "Order, cancel, order, cancel: Rapid Reset."

3.32 AI and DoS: AI-Powered DDoS and Attacks on AI Systems [not in the screenshots]

What it is (simple words): ADDED CONTEXT. The CEH v13 AI exam adds AI topics to every module. For this module there are two sides. First, attackers use AI to make DDoS stronger. Second, AI systems themselves can be the target of denial of service.

Analogy: Side one is hiring a smart planner for the mob. Side two is a mob that overloads a very expensive call center that charges per call.

How it works (numbered steps):

AI-powered attacks:

1. AI helps choose targets and weak points from open information.
2. AI writes or edits flooding scripts and phishing lures to build botnets.
3. AI adapts traffic in real time to look like human users and to avoid rate limits.

Attacks on AI systems:

1. In the OWASP Top 10 for LLM Applications 2023, the item was named "Model Denial of Service" (LLM04). The attacker sends resource-heavy prompts, very long inputs, or repeated requests.
2. In the 2025 list, this became LLM10 "Unbounded Consumption" (verify numbering against the current OWASP list).
3. The result is high cost, slow responses, or an unusable AI service.

Real-world use: A public chatbot that allows huge inputs and no limits gets thousands of long prompts per minute. The cloud bill grows and normal users see timeouts.

Tools and commands used here: API gateways, WAFs, usage quotas.

AI angle: Example prompt (defender): "Design rate limits for an LLM API: tokens per minute per user, max input size, and max output size."

Countermeasures and defense: Set input size limits, per-user rate limits and quotas, timeouts, caching, authentication, and cost alerts.

EXAM FOCUS:

- Keywords: "resource-heavy prompts", "model denial of service", "unbounded consumption" -> DoS against an LLM.
- Trap: it is not prompt injection (which changes behavior). It exhausts resources.

Memory trick: "Big prompts, big bills."

4. Tool Encyclopedia

Tools marked (A) are alternatives added by ADDED CONTEXT. Everything else appears in the screenshots.

4.1 Attack tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
ISB	Floods with HTTP, UDP, TCP, ICMP	GUI, Windows	Attack	"I'm so bored"
UltraDDOS-v2	Sends set number of packets to IP and port	GUI, Windows	Attack	"IP, port, number of packets"
LOIC	Simple GUI flood (TCP, UDP, HTTP)	GUI, Windows	Attack	"Low Orbit Ion Cannon"
HOIC	HTTP flood with booster scripts	GUI, Windows	Attack	"High Orbit Ion Cannon"
HULK	Unique obfuscated HTTP requests	Python script	Attack	"HTTP Unbearable Load King"
Slowloris	Holds connections with partial requests	Script	Attack	"partial HTTP requests"
UFONet	Uses web flaws to make sites hit the target	Python	Attack	"UFO" botnet via web apps
hping3 (A)	Crafts TCP, UDP, ICMP packets; floods	CLI, Linux	Scan and attack	"flag SYN, --flood"
Metasploit (A)	Has DoS auxiliary modules (for example synflood)	Framework	Attack	"auxiliary/dos"
slowhttptest (A)	Tests slow HTTP weaknesses	CLI, Linux	Attack, testing	"slow HTTP test"
T50 (A)	Mixed protocol packet injector	CLI, Linux	Attack	"flood many protocols"

4.2 Defense, detection, and protection tools

Tool or service	What it does (simple)	Type or platform	Phase used	Exam keyword
Anti DDoS Guardian	Protects Windows servers (IIS, mail, FTP, SIP)	Software	Defense	"IIS, Apache, Camfrog"
DDoS-Guard	DDoS protection service	Service	Defense	read from slide
DOSarrest	Cloud DDoS protection service	Cloud	Defense	"DOSarrest"
Radware DefensePro X	Attack mitigation device	Appliance	Defense	"DefensePro"
Gatekeeper	Open-source DDoS protection	Open source	Defense	github "Gatekeeper"
F5 DDoS Protection	Mitigation from F5	Appliance, cloud	Defense	"F5"
Cloudflare	Global network absorbs DDoS	Cloud	Defense	"100 Tbps"
Akamai DDoS Protection	Dedicated infrastructure and DNS protection	Cloud	Defense	"dedicated infrastructure"
Stormwall PRO	Cloud DDoS protection	Cloud	Defense	additional service
Imperva DDoS Protection	Cloud DDoS and WAF	Cloud	Defense	additional service
Nexusguard	Cloud DDoS protection	Cloud	Defense	additional service
BlockDoS	Cloud DDoS protection	Cloud	Defense	additional service
Blumira Honeypot	Detects access attempts and lateral movement	Software	Detection	"Blumira"
Cisco IPS	Reputation filtering of threat sources	Appliance	Defense	"source IP reputation"
Cisco IP Source Guard	Filters by DHCP snooping bindings	Switch feature	Defense	"IP source bindings"
Cisco TCP Intercept	Router protection from SYN flood	Router feature	Defense	"intercept and validate"

Tool or service	What it does (simple)	Type or platform	Phase used	Exam keyword
Wireshark (A)	Packet capture for attack analysis	GUI, all OS	Detection	"capture and inspect"
NetFlow collectors (A)	Flow statistics for baselining	Software	Detection	"flow data"

4.3 Look-alike tools and how to pick

- Attack tool or protection tool? LOIC, HOIC, HULK, Slowloris, UFONet, ISB, UltraDDOS-v2 attack. Gatekeeper, DOSarrest, Radware, F5, Cloudflare, Akamai protect.
- LOIC vs HOIC: Low Orbit = older, simple. High Orbit = HTTP focus with boosters.
- HULK vs Slowloris: HULK sends many complete unique requests (high volume). Slowloris sends few incomplete requests (low volume).
- hping3 vs nmap: nmap maps ports and services. hping3 crafts custom packets and floods.
- Honeypot vs IDS: a honeypot is bait. An IDS watches real traffic.

5. Command Sheet

The notes mention the ping command (Ping of Death) and the NTP monlist command by name. Other commands in this section are ADDED CONTEXT because the exam commonly asks them for this module (hping3) or because they implement the countermeasures on the slides (Cisco IOS).

5.1 ping (Ping of Death and ICMP flood)

Purpose: Send ICMP echo requests. The slide says Ping of Death uses "a simple ping command" with an oversized packet.

Full syntax:

```
ping -s <payload_bytes> -c <count> <target>      # Linux
ping -l <payload_bytes> -n <count> <target>      # Windows
```

Breakdown: the payload size sets the ICMP data size. Total packet size = payload + 8 bytes ICMP header + 20 bytes IP header. The slide example: 65,510 + 8 + 20 = 65,538 bytes, which is over the 65,535 limit.

Flag	Meaning	When to use	Example
-s (Linux)	Payload size in bytes	Test large packets	ping -s 1400 host
-l (Windows)	Payload size in bytes	Test large packets	ping -l 1400 host
-c (Linux)	Number of requests	Limit the test	ping -c 4 host

Flag	Meaning	When to use	Example
-n (Windows)	Number of requests	Limit the test	ping -n 4 host
-t (Windows)	Ping until stopped	Continuous test	ping -t host
-f (Linux)	Flood: send as fast as possible	Lab flood test, root only	ping -f host

Demo commands (lab, own host only):

```
ping -c 4 -s 1400 192.0.2.10
ping -c 1 -s 65507 192.0.2.10
ping -t 192.0.2.10
```

Sample output and how to read it:

```
PING 192.0.2.10 (192.0.2.10) 1400(1428) bytes of data.
1408 bytes from 192.0.2.10: icmp_seq=1 ttl=64 time=0.5 ms
1408 bytes from 192.0.2.10: icmp_seq=2 ttl=64 time=0.4 ms

--- 192.0.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
```

Read it top to bottom: the first line confirms the payload size that was actually sent (1400 bytes of data, 1428 with headers). Each reply line shows icmp_seq (which echo this is), ttl (hops left), and time (round-trip latency). "0% packet loss" at the bottom means the target answered every probe; a high loss percentage during a flood test means the target or the path is struggling.

Common mistakes: using -s larger than 65507 on Linux raises an error, because 65507 is the largest payload that still fits under the 65,535-byte IP limit (65,535 - 20 IP header - 8 ICMP header); forgetting -c and letting ping run forever; running -f without root privileges, which Linux blocks by default; testing a host you do not own or have written authorization to test, which can itself be a crime.

What the exam asks about it: matching "-s" or "-l" to "sets the payload size" and recognizing that a payload above 65,507 bytes on Linux cannot be sent in one unfragmented IP packet (RFC 791 limit), the exact scenario behind Ping of Death.

5.2 hping3 (SYN flood, UDP flood, ICMP flood, spoofing)

Purpose: Craft and send custom TCP, UDP, and ICMP packets for testing firewalls and simulating the floods in this module. Classic exam tool for the Denial-of-Service module. ADDED CONTEXT: not shown in the screenshots, but very commonly tested.

Full syntax:

```
hping3 [options] <target>
```

Flag table:

Flag	Meaning	When to use	Example
-S	Set the SYN flag	SYN flood simulation	hping3 -S target
-A	Set the ACK flag	ACK flood simulation	hping3 -A target
-R	Set the RST flag	Spoofed session testing	hping3 -R target
-F	Set the FIN flag	Spoofed session testing	hping3 -F target
--udp	Use UDP mode	UDP flood simulation	hping3 --udp target
--icmp	Use ICMP mode	ICMP flood simulation	hping3 --icmp target
-p	Destination port	Target a specific service	hping3 -S -p 80 target
--flood	Send as fast as possible, ignore replies	Load/flood testing	hping3 -S --flood target
--rand-source	Randomize (spoof) the source IP	Simulate many fake sources	hping3 --rand-source target
-a	Spoof one fixed source IP	Simulate a single spoofed sender	hping3 -a 10.0.0.5 target
-c	Number of packets to send	Limit a test	hping3 -S -c 1000 target
-i u	Delay between packets (microseconds)	Control send rate	hping3 -i u1000 target

Demo commands (lab, own target only):

```
hping3 -S -p 80 --flood --rand-source 192.0.2.10
hping3 --udp -p 53 --flood --rand-source 192.0.2.10
hping3 --icmp --flood -a 192.0.2.50 192.0.2.10
```

The first line simulates a SYN flood on port 80 with spoofed sources (section 3.12). The second simulates a UDP flood on port 53 (section 3.7). The third simulates an ICMP flood with one spoofed source, similar to the setup in a Smurf attack (section 3.10).

Sample output and how to read it:

```
HPING 192.0.2.10 (eth0 192.0.2.10): S set, 40 headers
len=46 ip=192.0.2.10 ttl=64 DF id=0 sport=80 flags=SA seq=0
len=46 ip=192.0.2.10 ttl=64 DF id=0 sport=80 flags=SA seq=1

--- 192.0.2.10 hping statistic ---
5000 packets transmitted, 0 packets received, 100% packet loss
```

Read it top to bottom: "S set" confirms SYN mode. Each reply line shows "flags=SA", meaning the target answered with SYN-ACK, which is normal behavior, not a sign the target is compromised. In a real flood with --flood and --rand-source, the attacker does not wait for or

read these replies, so hping3 reports "100% packet loss" at the end. That line means hping3 sent the packets and did not wait for real responses, not that the attack failed.

Common mistakes: forgetting --rand-source or -a and sending from your own real IP, which exposes the tester; running --flood on a production system without a signed authorization; confusing -S (SYN flag) with --udp or --icmp (different modes, cannot combine with -S); expecting --flood output to show meaningful statistics, since it is designed to not wait for replies.

What the exam asks about it: which flag sets which TCP flag (-S, -A, -R, -F); what --flood and --rand-source do; matching an hping3 command line to the attack it performs (SYN flood, UDP flood, ICMP flood, or a spoofed session flood using -A with -R or -F).

5.3 ntpdc (NTP monlist query, used in NTP amplification)

Purpose: Query an NTP server. The monlist sub-command (when enabled on the server) returns the list of recent clients, which is the data abused in an NTP amplification attack (section 3.11).

Full syntax:

```
ntpdc -n -c monlist <ntp_server>
```

Flag table:

Flag	Meaning	When to use	Example
-n	Show numeric addresses	Faster, avoids DNS lookups	ntpdc -n target
-c	Run one NTP control command	Query monlist	ntpdc -c monlist target

Demo command (lab or authorized audit only):

```
ntpdc -n -c monlist 192.0.2.20
```

Sample output and how to read it:

```
remote address        port local address     count
==============================================================
203.0.113.5           123 192.0.2.20         42
203.0.113.6           123 192.0.2.20         17
```

Read it top to bottom: each row is a client that recently talked to this NTP server. If this command works against a server you do not manage, that server is an open reflector and can be abused in an NTP amplification attack; report it to the operator.

Common mistakes: assuming monlist is safe because it is a normal NTP feature; it is the exact flaw abused in amplification. Running this against servers you do not own, without authorization, to probe them.

What the exam asks about it: recognizing that "monlist enabled" is the exam's shorthand for "this NTP server can be abused for amplification", and that the fix is to disable monlist or upgrade ntpd.

5.4 Cisco IOS: ip tcp intercept (SYN flood defense)

Purpose: Protect a TCP server from SYN flood attacks by having the router intercept and validate connection requests before they reach the server (section 3.25).

Full syntax:

```
ip tcp intercept list <acl-number-or-name>
ip tcp intercept mode {intercept | watch}
ip tcp intercept drop-mode {oldest | random}
ip tcp intercept max-incomplete {low | high} <number>
ip tcp intercept watch-timeout <seconds>
```

Flag table:

Flag/keyword	Meaning	When to use	Example
list	Which traffic to intercept (matches an ACL)	Scope protection to your servers	ip tcp intercept list 101
mode intercept	Router completes the handshake for the server	Strongest protection, more router load	ip tcp intercept mode intercept
mode watch	Router watches passively, resets slow half-opens	Lower router load	ip tcp intercept mode watch
drop-mode oldest	Drop the oldest half-open connection when full	Default-style behavior	ip tcp intercept drop-mode oldest
watch-timeout	Seconds to wait in watch mode before reset	Tune how fast watch mode reacts	ip tcp intercept watch-timeout 15

Demo commands (lab router):

```
Router(config)# access-list 101 permit tcp any host 192.0.2.10
Router(config)# ip tcp intercept list 101
Router(config)# ip tcp intercept mode intercept
```

Sample output and how to read it:

```
Router# show tcp intercept connections
Incomplete:
Client        Server        State     Create   Timeout
192.0.2.55:1050 192.0.2.10:80   SYNRCVD   00:00:02 00:00:28
```

Read it top to bottom: "Incomplete" lists half-open connections the router is holding on behalf of the server. "SYNRCVD" with a short "Create" time and a counting-down "Timeout" is normal traffic still completing the handshake. A screen full of many such entries with fake-looking client addresses points to an active SYN flood that TCP Intercept is absorbing instead of the server.

Common mistakes: forgetting the access list, so the feature protects nothing; choosing mode intercept on a very busy router without checking CPU headroom, since the router does the handshake work; confusing this feature with a firewall rule, when it specifically targets the TCP three-way handshake.

What the exam asks about it: matching "SYN flood defense on a router" to TCP Intercept, and knowing intercept mode means the router completes the handshake first.

5.5 Cisco IOS: ip verify unicast source reachable-via (ingress filtering, uRPF)

Purpose: Implement ingress filtering with Unicast Reverse Path Forwarding (uRPF): the router checks that a packet's source address is reachable through the interface it arrived on, and drops it if not. This is the router-level version of the ingress filtering described in section 3.25.

Full syntax:

```
ip verify unicast source reachable-via {rx | any}
  [allow-default] [allow-self-ping] [list <acl>]
```

Flag table:

Flag/keyword	Meaning	When to use	Example
rx	Strict mode: source must be reachable via this exact interface	Single-homed or symmetric routing	ip verify unicast source reachable-via rx
any	Loose mode: source must be reachable via any interface	Multi-homed, asymmetric routing	ip verify unicast source reachable-via any
allow-default	Allow a match against the default route	When no specific route covers the source	add allow-default
list	Exceptions that are always permitted or denied	Allow known special-case sources	list 110

Demo commands (lab router, edge interface facing customers):

```
Router(config)# interface GigabitEthernet0/1
Router(config-if)# ip verify unicast source reachable-via rx
```

Sample output and how to read it:

```
Router# show cef interface GigabitEthernet0/1 | include unicast
  IP unicast RPF check is enabled
```

Read it top to bottom: this single line confirms the feature is active on the interface. There is no per-packet log by default; dropped packets are counted in interface drop counters, which you check with "show interfaces" to see if spoofed traffic is being filtered.

Common mistakes: using rx (strict) mode on an interface with asymmetric routing, which drops legitimate traffic; forgetting that uRPF depends on Cisco Express Forwarding (CEF) being enabled; applying it only on one side of the network instead of at every customer-facing edge.

What the exam asks about it: matching "checks that the source address is reachable via the interface it arrived on" to Unicast RPF, and picking rx (strict) vs any (loose) based on whether routing is symmetric.

6. Ports, Protocols, Standards and Acronyms

6.1 Ports and protocols seen in the screenshots or implied by them

Protocol/Port	Simple purpose	Default port/layer	Known weakness / attack
ICMP	Error messages, diagnostics (ping)	Network layer (no port)	Flood, Smurf, Ping of Death
UDP	Connectionless transport	Transport layer (no handshake)	UDP flood, amplification base
TCP	Connection-oriented transport	Transport layer	SYN flood, SACK Panic, spoofed session
HTTP	Web requests	TCP port 80	GET/POST flood, Slowloris
HTTPS	Encrypted web requests	TCP port 443	Same app-layer floods, encrypted
NTP	Time synchronization	UDP port 123	Monlist amplification
DNS (ADDED CONTEXT)	Name resolution	UDP/TCP port 53	DNS amplification (same idea as NTP)

Protocol/Port	Simple purpose	Default port/layer	Known weakness / attack
HTTP/2 (ADDED CONTEXT)	Multiplexed web protocol	TCP port 443 usually	Rapid Reset (CVE-2023-44487)

6.2 Standards and acronyms

Term	Meaning	Why it matters here
DoS	Denial of Service	One-source availability attack
DDoS	Distributed Denial of Service	Many-source availability attack, uses a botnet
DRDoS	Distributed Reflection Denial of Service	Uses reflectors/secondary victims
C&C	Command and Control	Server that controls a botnet
CAM table	Content Addressable Memory table	Not DoS-specific; mentioned for contrast with Module 8
MSS	Maximum Segment Size	TCP option abused in SACK Panic
SACK	Selective Acknowledgement	TCP option abused in SACK Panic
SKB	Linux Socket Buffer	Kernel structure that overflowed in SACK Panic
RFC 791	IP standard	Defines the 65,535-byte packet limit (Ping of Death)
RFC 3704	Ingress filtering for multihomed networks	Botnet defense, drops spoofed/ bogus sources
BCP 38 (ADDED CONTEXT)	Network ingress filtering best practice	Related to RFC 3704, stops spoofing at the source network
uRPF	Unicast Reverse Path Forwarding	Router-level ingress filtering
PDoS	Permanent Denial of Service	Phlashing, bricking hardware
Cusum	Cumulative Sum algorithm	Used in sequential change-point detection
APK	Android Package	Format used to deliver mobile botnet malware
WAF (ADDED CONTEXT)	Web Application Firewall	Defends against application-layer floods
CDN (ADDED CONTEXT)	Content Delivery Network	Can absorb volumetric and app-layer load
OWASP LLM Top 10 (ADDED CONTEXT)	Security risks list for LLM apps	Covers model denial of service

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack and its countermeasure

Attack	Best-fit countermeasure
UDP flood	Rate limiting, block unused UDP ports, ISP scrubbing
ICMP flood	ICMP rate threshold, block unneeded ICMP
Ping of Death	Patch OS, drop oversized/malformed fragments
Smurf	Disable directed broadcast, ingress filtering
NTP amplification	Disable monlist, restrict NTP, ingress filtering
SYN flood	TCP Intercept, SYN cookies, larger backlog
Fragmentation attack	Limit reassembly buffers, drop bad fragments
Spoofed session flood	Strict stateful inspection, SYN proxy
HTTP GET/POST flood	Header/body timeouts, WAF, rate limiting
Slowloris	Connection limits, request timeouts, event-based server
Multi-vector	Layered defense across all three families
Permanent DoS (phlashing)	Signed firmware, secure boot, official sources only
TCP SACK Panic	Patch kernel, disable SACK as a workaround
DRDoS	Ingress filtering at ISPs, close open reflectors
Botnet growth (scanning/propagation)	Patch, segment network, egress filtering

7.2 Look-alike pairs

Pair	Difference in one line	How to tell them apart
DoS vs DDoS	One source vs many sources	Count distinct attacking IPs
DDoS vs DRDoS	Direct bots vs reflected traffic	Source looks like a real NTP or DNS server
Smurf vs NTP amplification	ICMP broadcast vs UDP monlist	Check the protocol: ICMP vs UDP/NTP
SYN flood vs ACK flood	Half-open SYNs vs ACK-only packets	See which TCP flag dominates the capture
SYN flood vs spoofed session	Only SYNs vs SYN/ACK/RST/FIN mix	Spoofed session traffic looks complete

Pair	Difference in one line	How to tell them apart
HTTP flood vs Slowloris	Many full requests vs few partial ones	Check bandwidth: normal vs very low
Ping of Death vs fragmentation	One oversized packet vs many fragments	Crash on reassembly vs slow resource drain
Permanent DoS vs normal DoS	Hardware damage vs temporary outage	Does service recover once attack stops
Egress vs ingress filtering	Checks outgoing vs incoming traffic	Direction of the check
Black hole vs TCP Intercept	Drops everything vs screens only SYNs	Black hole blocks all; Intercept is selective

8. Exam Cheat Sheet

8.1 Keyword to answer

- "non-legitimate service requests, reduces/restricts/prevents access" -> DoS attack definition.
- "botnet, zombies, C&C" -> DDoS.
- "pseudorandom permutation list" -> permutation scanning.
- "list of potentially vulnerable machines first" -> hit-list scanning.
- "own local network" -> local subnet scanning.
- "information obtained from an infected machine" -> topological scanning.
- "attack toolkit on a central source" -> central source propagation.
- "own system... copy transferred" -> back-chaining propagation.
- "at the exact time it breaks in" -> autonomous propagation.
- "bps" -> volumetric. "pps" -> protocol. "rps" -> application layer.
- "spoofed UDP, random ports, ICMP destination unreachable" -> UDP flood.
- "ICMP echo requests, reflection networks" -> ICMP flood.
- "malformed or oversized, exceeds 65,535 bytes" -> Ping of Death.
- "spoof victim IP, ICMP to broadcast network" -> Smurf.
- "monlist, NTP" -> NTP amplification.
- "fake source IP, SYN/ACK, listen queue, 75 seconds" -> SYN flood.
- "re-assemble fragmented packets, uninspected" -> fragmentation attack.
- "fake TCP session, SYN/ACK/RST/FIN" -> spoofed session flood.
- "time-delayed header" -> HTTP GET attack.
- "incomplete message body" -> HTTP POST attack.
- "partial HTTP requests, connection pool exhausted" -> Slowloris.
- "combination of attacks, changes form" -> multi-vector.
- "phlashing, bricking, hardware damage" -> permanent DoS.
- "SACK, MSS, integer overflow, SKB, kernel panic" -> TCP SACK Panic.
- "intermediary and secondary machines, reflect" -> DRDoS.

- "Cusum" -> sequential change-point detection.
- "spectral components" -> wavelet-based signal analysis.
- "RFC 3704" -> spoofed-address filtering for botnet defense.
- "headers of packets leaving" -> egress filtering.
- "prevents source spoofing" -> ingress filtering.
- "intercepting and validating TCP connection requests" -> TCP Intercept.
- "limited security system as bait" -> honeypot.
- "100 Tbps" -> Cloudflare.
- "HTTP/2, RST_STREAM" -> Rapid Reset.

8.2 Must-memorize lists

- Three attack families: Volumetric (bps), Protocol (pps), Application layer (rps).
- Five scanning methods: Random, Hit-list, Topological, Local subnet, Permutation.
- Three propagation methods: Central source, Back-chaining, Autonomous.
- Three detection techniques: Activity profiling, Sequential change-point detection, Wavelet-based signal analysis.
- Three countermeasure strategies: Absorb, Degrade, Shut down.
- Four "Prevent" controls: Egress filtering, Ingress filtering, TCP Intercept, Rate limiting.
- Four botnet-defense techniques: RFC 3704 filtering, Cisco IPS source IP reputation, Black hole filtering, DDoS prevention/IP Source Guard.

8.3 Numbers and defaults

- IP packet size limit (RFC 791): 65,535 bytes.
- Ping of Death example packet: 65,538 bytes (20 IP + 8 ICMP + 65,510 data).
- SYN listen queue hold time: at least 75 seconds.
- TCP SACK Panic MSS value used by the attacker: 48 bytes (the lowest value).
- NTP port: UDP 123 (ADDED CONTEXT, verify).
- Cloudflare infrastructure figure from the slide: 100 Tbps (verify current figure).
- Google Cloud HTTP/2 Rapid Reset peak (ADDED CONTEXT, verified by web search): above 398 million requests per second, August 2023, disclosed 10 October 2023.

8.4 Rules for picking the BEST option

- If bandwidth (bps) is the bottleneck, think volumetric and upstream/ISP mitigation. If connections/state tables are the bottleneck, think protocol and TCP Intercept. If app behavior (rps) is the bottleneck, think application layer and WAF/rate limiting.
- If the question says the attack fills the WAN link, a local firewall fix is not the best answer; pick ISP or cloud scrubbing.
- If the question says "few handlers control many agents", the best next step is to neutralize the handlers, not chase every agent.
- Shutting down services is the last-resort answer, not the first choice, unless the scenario demands an immediate stop no matter the cost.

- When two options both sound like filtering, check direction: egress (leaving) protects others; ingress (entering) protects you.

9. Practice Questions

These are original practice questions written for this guide. They are not taken from any real exam.

Q1. A company's web server becomes unreachable. Analysts see bandwidth at 95 percent of capacity, filled with UDP traffic from thousands of source IPs to random ports. Which attack family is this?

A. Protocol attack B. Application layer attack C. Volumetric attack D. Permanent DoS attack

Answer: C. The description matches "consumes bandwidth" and is measured in bps, the definition of a volumetric attack (section 3.6). A is wrong because protocol attacks exhaust connection state tables, not raw bandwidth. B is wrong because application layer attacks target app resources (rps), not bandwidth. D is wrong because permanent DoS damages hardware, not bandwidth.

Q2. An infected host searches only its own local network to find more systems to infect. Which scanning method is this?

A. Hit-list scanning B. Permutation scanning C. Local subnet scanning D. Topological scanning

Answer: C. The slide defines local subnet scanning exactly this way: "the infected machine looks for new vulnerable machines in its own local network" (section 3.3). A is wrong because hit-list scanning uses a pre-collected list, not the local network. B is wrong because permutation scanning uses a shared pseudorandom address list. D is wrong because topological scanning uses data found on the infected host (such as contacts), not a network scan of the local subnet.

Q3. Which propagation technique has the attacking host itself transfer the toolkit to the victim at the exact moment of exploitation?

A. Central source propagation B. Back-chaining propagation C. Autonomous propagation D. Hit-list propagation

Answer: C. The slide's own wording for autonomous propagation is "the attacking host itself transfers the attack toolkit... at the exact time that it breaks into that system" (section 3.4). A is wrong because central source uses a separate server to hold the toolkit. B is wrong because back-chaining transfers from the attacker's own system, not at the moment of exploitation specifically tied to the attacking host. D is a distractor; "hit-list" is a scanning method, not a propagation method.

Q4. An attacker spoofs a victim's IP address and sends ICMP echo requests to the broadcast address of a network with many live hosts. What is this attack called?

A. ICMP flood B. Smurf attack C. NTP amplification D. Ping of Death

Answer: B. Spoofing the victim's IP and sending to a broadcast address so many hosts reply to the victim is the definition of a Smurf attack (section 3.10). A is wrong because a plain ICMP flood does not require a broadcast address or spoofing. C is wrong because NTP amplification uses UDP and the monlist command, not ICMP broadcast. D is wrong because Ping of Death relies on an oversized packet, not a broadcast reflection.

Q5. A penetration tester runs the following and the target never completes the handshake. Which attack is being simulated?

```
hping3 -S -p 80 --flood --rand-source 192.0.2.10
```

A. UDP flood B. SYN flood C. ACK flood D. Slowloris

Answer: B. The -S flag sets the SYN flag, --flood sends packets as fast as possible, and --rand-source spoofs the source so replies never return, exactly the SYN flood pattern in section 3.12 and the hping3 sheet in section 5.2. A is wrong because --udp is not used here. C is wrong because -A, not -S, sets the ACK flag. D is wrong because Slowloris is an application-layer tool that sends partial HTTP requests, not raw SYN packets.

Q6. What does the --rand-source flag do in hping3?

A. Sends packets to random destination ports B. Spoofs a different random source IP on each packet C. Randomizes the TCP window size D. Picks a random protocol for each packet

Answer: B. --rand-source is the flag that randomizes (spoofs) the source IP address of each packet (section 5.2). A describes a different option used for scanning random ports, not --rand-source. C and D describe features hping3 does not perform with this flag.

Q7. An NTP server has the monlist command enabled. An attacker spoofs the victim's IP and sends small requests to many such servers. What is this attack, and what is the main risk factor?

A. Smurf attack; risk factor is the broadcast address B. NTP amplification; risk factor is monlist producing large responses C. SYN flood; risk factor is the listen queue D. Fragmentation attack; risk factor is packet reassembly

Answer: B. The slide ties NTP amplification directly to the monlist command, which turns a small request into a large response sent to the spoofed (victim) address (section 3.11). A is wrong because Smurf uses ICMP and a broadcast address, not NTP. C and D describe unrelated attacks with different mechanisms.

Q8. During an attack, a Linux server suddenly suffers a kernel panic. Logs show a stream of TCP packets with SACK options and an MSS value of 48 bytes. What is this attack, and what kernel structure is involved?

A. SYN flood; listen queue B. TCP SACK Panic; Linux Socket Buffer (SKB) C. Fragmentation attack; reassembly buffer D. Spoofed session flood; connection table

Answer: B. MSS forced to the lowest value (48 bytes) combined with SACK packets causing an integer overflow in the SKB matches the module's description of TCP SACK Panic exactly (section 3.19). A, C, and D describe different attacks with different mechanisms and do not involve the SKB integer overflow.

Q9. A company's router is configured with:

```
ip tcp intercept mode intercept
ip tcp intercept list 101
```

What does "mode intercept" cause the router to do?

A. Silently drop all TCP SYN packets B. Complete the TCP handshake with the client before connecting to the server C. Forward every packet to a honeypot for inspection D. Block all UDP traffic on the interface

Answer: B. In intercept mode, the TCP Intercept feature answers the client's SYN itself and only opens a connection to the real server once the client completes the handshake, protecting the server from SYN floods (section 5.4). A is wrong because the router answers SYNs rather than dropping them all. C describes honeypot deflection, a different control (section 3.26). D is unrelated to TCP Intercept, which only affects TCP.

Q10. Which command and flag combination would reveal whether a public NTP server is vulnerable to abuse in an amplification attack?

A. `ping -s 65507 ntp-server` B. `ntpdc -c monlist ntp-server` C. `hping3 --icmp ntp-server` D. `nmap -sS -p 80 ntp-server`

Answer: B. Querying monlist is exactly how a tester checks whether an NTP server will return the list of recent clients, the flaw abused in NTP amplification (section 5.3). A tests packet size limits (Ping of Death), not NTP abuse. C sends an ICMP flood, unrelated to NTP. D scans an unrelated TCP port.

Q11. A victim organization is hit by traffic that appears to come from thousands of legitimate-looking NTP and DNS servers, not from the attacker's own botnet IPs directly. What is this pattern called?

A. DDoS B. DRDoS C. Permanent DoS D. Multi-vector attack

Answer: B. Traffic reflected and amplified through innocent intermediary and secondary servers, so the target is hit by those servers rather than the attacker, is the definition of DRDoS (section 3.20). A is wrong because plain DDoS traffic comes directly from bots, not reflectors. C and D describe unrelated attack types.

Q12. Which of the following best describes the difference between egress filtering and ingress filtering?

A. Egress filtering checks traffic leaving the network; ingress filtering checks traffic entering the network B. Egress filtering blocks UDP; ingress filtering blocks TCP C. Egress filtering is done by the ISP only; ingress filtering is done only by the victim D. They are two names for the same control

Answer: A. The module defines egress filtering as scanning headers of packets leaving a network, and ingress filtering as preventing source address spoofing of traffic entering a network (section 3.25). B, C, and D misstate the actual distinction, which is about direction, not protocol or who performs it.

Q13. An attacker builds a mobile botnet by hiding malware inside a modified APK and distributing it through a third-party app store. What category of attack preparation does this represent?

A. Protocol attack B. Mobile device recruitment into a botnet C. TCP SACK Panic D. Black hole filtering

Answer: B. This matches the module's description of using mobile devices as DDoS botnets through malicious APKs (section 3.5). A and C are unrelated DoS techniques. D is a defensive control, not an attack preparation step.

Q14. During an active DDoS attack, security staff decide to keep the payment and login services running while turning off the product recommendation engine and image gallery. Which countermeasure strategy is this?

A. Absorbing the attack B. Degrading services C. Shutting down the services D. Black hole filtering

Answer: B. Keeping critical services up while stopping non-critical ones is the exact definition of "degrading services" (section 3.23). A is wrong because absorbing means adding capacity, not turning services off. C is wrong because shutting down means stopping everything. D is an unrelated routing-level control.

Q15. Which detection technique uses the Cusum algorithm to locate DoS attacks in network traffic statistics?

A. Activity profiling B. Sequential change-point detection C. Wavelet-based signal analysis D. Black hole filtering

Answer: B. The module ties the Cusum algorithm specifically to sequential change-point detection (section 3.22). A uses average packet rates and flow clusters, not Cusum. C analyzes spectral components. D is a countermeasure, not a detection technique.

Q16. A victim's incident responders find and disable the few C&C handler servers controlling a botnet of 50,000 agents. Why is this an efficient countermeasure?

A. It directly patches every infected agent B. There are usually far fewer handlers than agents, so disabling handlers can render many agents useless C. It is a form of black hole filtering D. It prevents mobile devices from being infected in the future

Answer: B. The module states this directly: there are usually fewer DDoS handlers than agents, so neutralizing a few handlers can render multiple agents useless (section 3.24). A is wrong because disabling handlers does not patch agent machines. C and D describe unrelated controls.

Q17. Which attack is best described as "irreversible damage to system hardware caused by a fraudulent firmware update"?

A. TCP SACK Panic B. Fragmentation attack C. Permanent Denial-of-Service (phlashing) D. DRDoS

Answer: C. Phlashing/permanent DoS is defined in the module as exactly this: hardware sabotage requiring replacement or reinstall, via fraudulent hardware updates (section 3.18). A, B, and D describe temporary network or kernel-level disruptions, not hardware damage.

Q18. What is the main difference between an HTTP GET attack and an HTTP POST attack as described in this module?

A. GET attacks use UDP; POST attacks use TCP B. GET attacks exploit an incomplete header; POST attacks exploit an incomplete message body C. GET attacks are volumetric; POST attacks are protocol attacks D. GET attacks target DNS; POST attacks target NTP

Answer: B. The module specifically distinguishes them this way: GET uses a time-delayed header, POST sends complete headers but an incomplete body (section 3.15). A, C, and D misstate the protocols and categories involved.

Q19. A tester observes a single attacking machine sending very low bandwidth traffic, but the target web server's maximum concurrent connection pool is completely exhausted. Which attack is most likely?

A. UDP flood B. NTP amplification C. Slowloris D. Smurf attack

Answer: C. Low bandwidth combined with an exhausted connection pool is the signature of Slowloris, which uses partial HTTP requests to hold connections open cheaply (section 3.16). A, B, and D all require or produce much higher traffic volume.

Q20. Which of the following is an example of a volumetric attack technique according to the module's categorization?

A. SYN flood B. HTTP GET/POST attack C. NTP amplification attack D. TCP SACK Panic attack

Answer: C. The module lists NTP amplification under volumetric attacks (section 3.6). A and D are listed under protocol attacks. B is listed under application layer attacks.

Q21. An attacker uses a publicly available GUI tool where they only need to enter the target IP address, a port, and the number of packets to transmit. Which tool matches this exact description?

A. Slowloris B. UltraDDOS-v2 C. hping3 D. Blumira

Answer: B. The module describes UltraDDOS-v2 exactly this way: a simple GUI where attackers enter target IP, port, and the number of packets to send (section 3.21). A is a different tool with no such GUI fields described. C is a command-line tool, not GUI-only. D is a defensive honeypot tool, not an attack tool.

Q22. Which Cisco IOS feature uses the DHCP snooping binding database to prevent a bot from succeeding with spoofed packets?

A. TCP Intercept B. IP Source Guard C. Black hole filtering D. RFC 3704 filtering

Answer: B. The module ties IP Source Guard directly to the DHCP snooping binding database as a defense against spoofed packets (section 3.27). A defends against SYN floods specifically. C silently drops traffic at the routing level without using DHCP bindings. D filters spoofed or bogus source addresses at the ISP level, independent of DHCP snooping.

Q23. A DDoS protection vendor advertises "100 Tbps of infrastructure" to absorb attacks. Which service does the module associate with this figure?

A. Akamai B. Cloudflare C. Imperva D. BlockDoS

Answer: B. The module's slide states this figure specifically for Cloudflare (section 3.30). A, C, and D are listed as other protection services without this specific figure in the notes.

Q24. In 2023, Google, Cloudflare, and AWS disclosed a record-setting DDoS technique using HTTP/2 stream cancellation. What is this attack called, and roughly how large was the peak reported by Google?

A. Rapid Reset; about 398 million requests per second B. Slowloris; about 50,000 requests per second C. Smurf; about 10 Gbps D. SACK Panic; about 1 million packets per second

Answer: A. This is the HTTP/2 "Rapid Reset" attack (CVE-2023-44487), and Google reported a peak above 398 million requests per second in August 2023 (section 3.31, verified by web search). B, C, and D name unrelated attacks with very different scales and mechanisms.

Q25. Why is Rapid Reset classified as an application-layer (layer 7) attack rather than a volumetric attack?

A. It is measured in bits per second, like a UDP flood B. It is measured in requests per second, exhausting server-side request processing C. It relies on spoofed ICMP broadcast traffic D. It requires a botnet of mobile phones

Answer: B. Rapid Reset abuses HTTP/2 stream resets to force the server to process many requests cheaply from the client's side, matching the rps-based application layer definition in

section 3.6. A describes volumetric attacks. C describes Smurf. D describes mobile botnets, unrelated to the mechanism here.

Q26. Which best-practice rule should guide the choice between TCP Intercept and a Web Application Firewall (WAF) for a given DoS complaint?

A. Always choose TCP Intercept, since it is newer B. Match the control to the attack family: TCP Intercept for protocol/SYN-based attacks, WAF for application-layer attacks C. Always choose a WAF, since it covers every attack family D. Use black hole filtering instead of either

Answer: B. The module's layered-defense logic (sections 3.6 and 8.4) ties each control to the family it protects: TCP Intercept guards the TCP handshake (protocol attacks), while a WAF inspects HTTP-level behavior (application-layer attacks). A and C oversimplify by ignoring attack family. D answers a different question (how to stop traffic to one IP entirely, which also drops legitimate traffic).

Q27. A security engineer wants to stop internally compromised hosts from launching spoofed-source floods against other organizations. Which control is most appropriate to deploy on the organization's own outbound traffic?

A. Ingress filtering B. Egress filtering C. TCP Intercept D. NTP monlist disabling

Answer: B. Egress filtering specifically checks and restricts traffic leaving the organization's own network, stopping spoofed-source floods from leaving in the first place (section 3.25). A protects the organization from incoming spoofed traffic, not outgoing. C protects a server from SYN floods, unrelated to outbound spoofing. D is a server-side NTP configuration, not a network egress control.

Q28. [AI question] A defender notices a public chatbot API is receiving extremely long prompts at a high rate from a small number of accounts, driving up compute cost and slowing responses for all users. Which concept from the OWASP LLM Top 10 best matches this?

A. Prompt injection B. Model Denial of Service / Unbounded Consumption C. Data poisoning D. Insecure plugin design

Answer: B. Resource-heavy prompts driving up cost and degrading availability is the definition of Model Denial of Service, later renamed Unbounded Consumption in the OWASP LLM Top 10 (section 3.32). A describes attempts to change model behavior through crafted input, not resource exhaustion. C describes tampering with training data. D describes insecure integrations, unrelated to resource exhaustion.

Q29. [AI question] Which defensive control would most directly reduce the risk described in Q28?

A. Disabling TLS on the API B. Per-user rate limits and maximum input/output token sizes C. Switching from UDP to TCP for the API D. Enabling ICMP redirects

Answer: B. Token and rate limits are the module's recommended defense against AI resource-exhaustion DoS (section 3.32). A weakens security without addressing resource exhaustion. C and D are unrelated network-layer changes that do not limit prompt size or request rate.

Q30. A network team captures packets showing many TCP segments with the ACK flag set, no prior SYN seen for those flows, and no matching entries in the firewall's session table. The firewall appears to pass most of them anyway. Which attack best fits this evidence?

A. UDP flood B. Spoofed session flood (multiple ACK variant) C. Ping of Death D. Permanent DoS

Answer: B. ACK-heavy traffic with no prior SYN, built to look like an established session and bypass a firewall, matches the "multiple ACK spoofed session flood" variant described in section 3.14. A is UDP-based with no TCP flags involved. C involves oversized ICMP packets, not TCP ACKs. D causes hardware damage, not a traffic pattern like this.

10. Flashcards

Q: What does DoS stand for and what does it attack? | A: Denial of Service; it attacks availability, not confidentiality or integrity. Q: What turns a DoS into a DDoS? | A: Using many compromised machines (a botnet) controlled by a C&C server instead of one source. Q: Name the three DoS/DDoS attack vector families. | A: Volumetric (bps), Protocol (pps), Application layer (rps). Q: Which scanning method uses a shared pseudorandom IP list? | A: Permutation scanning. Q: Which scanning method pre-collects a list of likely targets? | A: Hit-list scanning. Q: Which scanning method uses data from the infected machine itself? | A: Topological scanning. Q: Name the three malicious code propagation techniques. | A: Central source, Back-chaining, Autonomous. Q: What makes autonomous propagation different? | A: The attacking host transfers the toolkit at the exact moment it exploits the victim. Q: What is a UDP flood's telltale server response? | A: ICMP "Destination Unreachable" errors for non-existent application ports. Q: What triggers an ICMP flood protection feature? | A: A configured threshold limit on ICMP echo requests per second being exceeded. Q: What packet size limit does Ping of Death exceed? | A: 65,535 bytes, the IP packet size limit from RFC 791. Q: How does a Smurf attack use IP spoofing? | A: It spoofs the source as the victim's IP and sends ICMP echo requests to a broadcast address. Q: What NTP command is abused in NTP amplification? | A: monlist. Q: How long must a host hold a half-open SYN connection, per the module? | A: At least 75 seconds. Q: Why does SYN flooding work? | A: It exploits the small size of the TCP listen queue and the handshake's wait for a final ACK. Q: What resource does a fragmentation attack mainly exhaust? | A: CPU and memory spent reassembling and inspecting large fragmented packets. Q: What are the two spoofed session flood variants? | A: Multiple SYN-ACK variant and multiple ACK variant (both with RST or FIN). Q: What does an HTTP GET attack delay? | A: The HTTP request header, using a time-delayed header. Q: What does an HTTP POST attack leave incomplete? | A: The message body. Q: What makes Slowloris effective with low bandwidth? | A: It holds many connections open with partial requests until the connection pool is exhausted. Q: What is a multi-vector DDoS attack? | A: A

combination of volumetric, protocol, and application-layer attacks, launched in sequence or in parallel. Q: What is another name for Permanent Denial-of-Service? | A: Phlashing, also called "bricking a system". Q: What vulnerability class does TCP SACK Panic exploit? | A: An integer overflow in the Linux Socket Buffer (SKB) triggered by SACK packets with a very low MSS. Q: What MSS value does the TCP SACK Panic attack use? | A: 48 bytes, the lowest value. Q: What is the key difference between DDoS and DRDoS? | A: DDoS bots attack directly; DRDoS traffic is reflected through intermediary and secondary victim servers. Q: Name the three DoS/DDoS detection techniques. | A: Activity profiling, Sequential change-point detection (Cusum), Wavelet-based signal analysis. Q: Name the three countermeasure strategies. | A: Absorbing the attack, Degrading services, Shutting down the services. Q: What does egress filtering check? | A: Headers of IP packets leaving a network, to stop malicious traffic from exiting. Q: What does ingress filtering check? | A: Incoming traffic, to prevent source address spoofing. Q: What does TCP Intercept do? | A: Intercepts and validates TCP connection requests to protect servers from SYN floods. Q: What is a honeypot used for in this module? | A: A low-security system that entices attackers and records their techniques, for deflecting DoS traffic. Q: What does RFC 3704 filtering target? | A: Traffic from unused or reserved (bogus) IP addresses, filtered at the ISP. Q: What does black hole filtering do? | A: Discards packets at the routing level without informing the sender. Q: What Cisco feature uses the DHCP snooping binding database against spoofed packets? | A: IP Source Guard. Q: What figure does the module give for Cloudflare's infrastructure? | A: 100 Tbps (verify current figure). Q: What is the HTTP/2 Rapid Reset attack's mechanism? | A: Rapidly opening and cancelling (RST_STREAM) HTTP/2 streams so the server does work the client never waits for. Q: What was the peak rps reported by Google for Rapid Reset? | A: Above 398 million requests per second (August 2023). Q: What hping3 flag sets the SYN flag? | A: -S. Q: What hping3 option spoofs a random source IP per packet? | A: --rand-source. Q: What does Unicast RPF (uRPF) check? | A: That a packet's source address is reachable via the interface it arrived on. Q: In the OWASP LLM Top 10, what risk matches resource-exhausting prompts? | A: Model Denial of Service, later called Unbounded Consumption.

11. Glossary

ACK flood: a flood of TCP packets with only the ACK flag set, aimed at connection state tables. Activity profiling: a detection technique based on average packet rates and network flow clusters. Amplification attack: an attack where a small request produces a much larger response sent to a spoofed victim. APK: Android Package, the file format for Android apps, abused to build mobile botnets. Autonomous propagation: malicious code transfer that happens at the exact moment of exploitation, from the attacking host itself. Back-chaining propagation: malicious code transfer from the attacker's own system to a newly found victim. Black hole filtering: discarding packets at the routing level without notifying the sender. Botnet: a network of compromised machines (bots or zombies) controlled by an attacker. BrickerBot (ADDED CONTEXT): malware that permanently damages poorly secured IoT devices. C&C (Command and Control): the server or infrastructure an attacker uses to control a botnet. Central source propagation: malicious code transfer from one shared server to new victims. Cusum (Cumulative Sum): an algorithm used in sequential change-point detection to

spot DoS attacks. DDoS (Distributed Denial of Service): a DoS attack launched from many compromised sources. DNS amplification (ADDED CONTEXT): an amplification attack using open DNS resolvers, similar to NTP amplification. DoS (Denial of Service): an attack that reduces, restricts, or prevents access to a resource for legitimate users. DRDoS (Distributed Reflection Denial of Service): a DDoS variant that reflects traffic through intermediary and secondary victims. Egress filtering: scanning and restricting IP packet headers leaving a network. Flash event: a sudden but legitimate surge in traffic, which detection systems must tell apart from an attack. Fragmentation attack: flooding a target with TCP or UDP fragments that exhaust reassembly and inspection resources. Handler: a DDoS control node that issues commands to agents (bots). Hit-list scanning: scanning a pre-collected list of likely-vulnerable hosts. Honeypot: a low-security decoy system used to attract and study attackers. HTTP GET attack: an application-layer attack using a time-delayed HTTP header. HTTP POST attack: an application-layer attack using an incomplete HTTP message body. ICMP flood: a flood of ICMP echo request packets aimed at saturating a victim's bandwidth and CPU. Ingress filtering: filtering incoming traffic to prevent source address spoofing. IP Source Guard: a Cisco switch feature that filters traffic using the DHCP snooping binding database. Local subnet scanning: a worm scanning method that searches the infected host's own local network. Monlist: an NTP command that returns recent client addresses, abused in NTP amplification. MSS (Maximum Segment Size): the largest TCP segment size a host will accept, abused with a very low value in TCP SACK Panic. Multi-vector attack: a DDoS attack combining volumetric, protocol, and application-layer techniques. NTP amplification: an attack that abuses the NTP monlist command to flood a spoofed victim with large responses. OWASP LLM Top 10 (ADDED CONTEXT): a list of top security risks for large language model applications, including model denial of service. Permanent Denial of Service (PDoS, phlashing): an attack that causes irreversible hardware damage, requiring replacement. Ping of Death: an oversized or malformed ping packet that crashes a target during reassembly. Protocol attack: a DoS family that exhausts connection state tables, measured in pps. Pulse wave attack: short, repeated high-volume bursts with quiet gaps between them. Rapid Reset (CVE-2023-44487) (ADDED CONTEXT): an HTTP/2 attack that rapidly opens and cancels streams to overload servers. Rate limiting: controlling the rate of traffic allowed on an interface. Reflection attack: an attack where requests are sent to an innocent third party, which replies to a spoofed victim. RFC 3704: a filtering standard that denies traffic from unused or reserved IP addresses. SACK (Selective Acknowledgement): a TCP option letting a receiver acknowledge specific received segments, abused in TCP SACK Panic. Sequential change-point detection: a detection technique using the Cusum algorithm to spot sudden traffic changes. SKB (Linux Socket Buffer): the kernel data structure whose integer overflow caused the TCP SACK Panic crash. Slowloris: a low-bandwidth attack that holds many connections open with partial HTTP requests. Smurf attack: a spoofed ICMP broadcast attack that floods a victim with replies from many hosts. Spoofed session flood attack: a fake TCP session built from SYN, ACK, RST, and FIN packets to bypass firewalls. SYN flood: a flood of fake-source SYN packets that fills a target's half-open connection queue. TCP Intercept: a Cisco router feature that validates TCP connection requests to protect servers from SYN floods. TCP SACK Panic: a Linux kernel crash caused by SACK packets with a very low MSS, exploiting an SKB integer overflow. Topological scanning: a worm scanning method using

data found on an already-infected host. Unbounded Consumption (ADDED CONTEXT): the 2025 OWASP LLM Top 10 name for resource-exhaustion attacks on AI models. Unicast RPF (uRPF): a router feature that verifies a packet's source is reachable via the interface it arrived on. Volumetric attack: a DoS family that exhausts bandwidth, measured in bps. Wavelet-based signal analysis: a detection technique that examines traffic as spectral components to find anomalies. Zero-day DDoS attack: a DDoS method using a new, unpatched flaw with no existing signature. Zombie (bot): a compromised machine controlled remotely as part of a botnet.

12. Coverage Report

12.1 Map of pages to guide sections

Page	Slide topic	Guide section(s)
1	What is a DoS attack	3.1
1	What is a DDoS attack	3.2
2	Scanning methods for finding vulnerable machines	3.3
2	How does malicious code propagate	3.4
3	Mobile devices as botnets for DDoS	3.5
3	Basic categories of DoS/DDoS attack vectors	3.6
4	UDP flood attack	3.7
4	ICMP flood attack	3.8
5	Ping of Death and Smurf attacks	3.9, 3.10
5	NTP amplification attack	3.11
6	SYN flood attack	3.12
6	Fragmentation attack	3.13
7	Spoofed session flood attack	3.14
7	HTTP GET/POST and Slowloris attacks	3.15, 3.16
8	Multi-vector attack	3.17
8	Permanent DoS and TCP SACK Panic attack	3.18, 3.19
9	Permanent DoS and TCP SACK Panic (repeat)	3.19

Page	Slide topic	Guide section(s)
9	Distributed Reflection DoS (DRDoS) attack	3.20
10	DoS/DDoS attack toolkits in the wild	3.21
10	Detection techniques	3.22
11	DoS/DDoS countermeasure strategies	3.23
11	DDoS attack countermeasures (secondary victims, handlers)	3.24
12	DDoS countermeasures: prevent potential attacks	3.25
12	DDoS countermeasures: deflect attacks (honeypots)	3.26
13	Techniques to defend against botnets	3.27
13	DoS/DDoS protection at ISP level	3.28
14	DoS/DDoS protection tools	3.29
14	DoS/DDoS protection services	3.30
15	Module summary, botnet ecosystem, Rapid Reset case study	3.31
(exam topic, no screenshot)	AI-powered DDoS and DoS against AI systems	3.32

12.2 Items marked [UNCLEAR]

- Page 1: exact wording of DDoS ecosystem diagram steps 8 to 11 (malware-to-C&C and attack-trigger steps).
- Page 2: the exact row labels for "hit-list scanning" and "topological scanning" in the five-method table; names inferred from the matching descriptions.
- Page 3: the sentence about the attacker "binding the malicious APK server to the APK file" is oddly worded on the slide; the likely intended meaning (binding a payload into a legitimate app's APK) is noted.
- Page 13: the exact bandwidth figures and bot counts shown in the ISP-level protection diagram (small text on the slide).
- Page 14: the "Newsguard" entry under DDoS protection services is likely a misread of "Nexusguard"; both names are given, with a note to verify.

12.3 Items to double check

- Verify the current Cloudflare infrastructure capacity figure (the slide says 100 Tbps; this number changes over time).
- Verify the exact monlist response size and historical amplification factor for NTP amplification attacks (ADDED CONTEXT estimate given).
- Verify the exact OWASP LLM Top 10 risk ID for model denial of service in the current published version (it moved from LLM04 in 2023 to Unbounded Consumption, LLM10, in 2025; always check the latest list).
- Verify current Cisco IOS command syntax for ip tcp intercept and ip verify unicast source reachable-via against the specific IOS or IOS XE version in use, since options vary by release.
- Verify the TCP SACK Panic CVE identifiers (CVE-2019-11477, CVE-2019-11478, CVE-2019-11479) and confirm which kernel versions are patched for any system you manage.

12.4 Coverage confirmation

Every line in the page-by-page inventory gathered while building this guide (all 15 pages, 29 slide images) is placed in exactly one section of this guide: concepts and ecosystem in 3.1 to 3.6, each named attack in 3.7 to 3.20, tools and detection in 3.21 to 3.22, countermeasures and strategies in 3.23 to 3.30, and the module summary plus the two items it references but did not show (botnet ecosystem detail and the Google Cloud Rapid Reset case study) in 3.31. The AI angle required by the exam facts is covered per topic and in the dedicated section 3.32. Tools are cross-referenced in section 4, commands in section 5, and every acronym or standard introduced appears in section 6 and the glossary (section 11). No item from the inventory was dropped.